
ACTIVIDAD 11

25 DE MAYO DE 2026
UNIVERSIDAD ANTONIO NARIÑO

ACTIVIDAD 11: PLAN DE PRUEBAS DE ACEPTACIÓN

Luis Carlos Rojas Criollo

**Universidad Antonio Nariño
FACULTAD DE SISTEMAS**

**Desarrollo de Aplicaciones Seguras
ESPECIALIZACIÓN EN CIBERSEGURIDAD
Mayo 2026.**

Contenido

Introducción	3
Objetivo	3
Diseño Casos de Prueba en TestLink	3
Casos de Prueba para Registro de Nuevos pacientes.....	6
Casos de Prueba para Búsqueda de Pacientes	8
Casos de Prueba para Actualización de Información de Pacientes.....	9
Casos de Prueba para Registro de Historial Médico	11
Asignar Casos al Plan de Pruebas	13
Ejecutar y Registrar Resultados en TestLink	14
Ejecutar Pruebas de Interfaz de Usuario con Selenium	14
Preparación del Entorno Selenium.....	14
Prueba Selenium de Login y Roles	16
Prueba Selenium para Búsqueda/Consulta de Pacientes	17
Prueba Selenium de Validaciones y Mensajes de Error	18
Prueba Selenium para Historial Médico	19
Pruebas de API con Postman	20
Preparación de Postman para Pruebas de API	20
Creación de Endpoints API en Laravel.....	21
Endpoints de Pacientes en Postman	22
Endpoints de Historial Médico en Postman	25
Endpoints de Doctores en Postman	28
Pruebas de seguridad y manejo de errores en Postman	29
Exportar la Colección de Postman para GitHub.....	33
Exploración y Pruebas de Flujos Realistas Combinando Frontend y Backend	33
Pruebas De Volumen Con Base De Datos Llena	37
Pruebas De Seguridad y Permisos.....	39
Smoke Test Suite Crítico	43
Pruebas de Compatibilidad con Navegadores.....	44
Pruebas de Carga y Estrés con JMeter	45
Evaluación por Usuario Final	46
Link GitHub	46
Resultado de aprendizaje.....	47
Referencias	48

Introducción

La presente actividad tiene como finalidad desarrollar un plan de pruebas de aceptación para una aplicación web de gestión de pacientes clínicos. Para ello, se validan diferentes componentes del sistema, como los casos de prueba en TestLink, pruebas de interfaz con Selenium, pruebas de API con Postman, flujos realistas entre frontend y backend, pruebas de volumen, seguridad, compatibilidad, carga, estrés y una simulación de evaluación por usuario final.

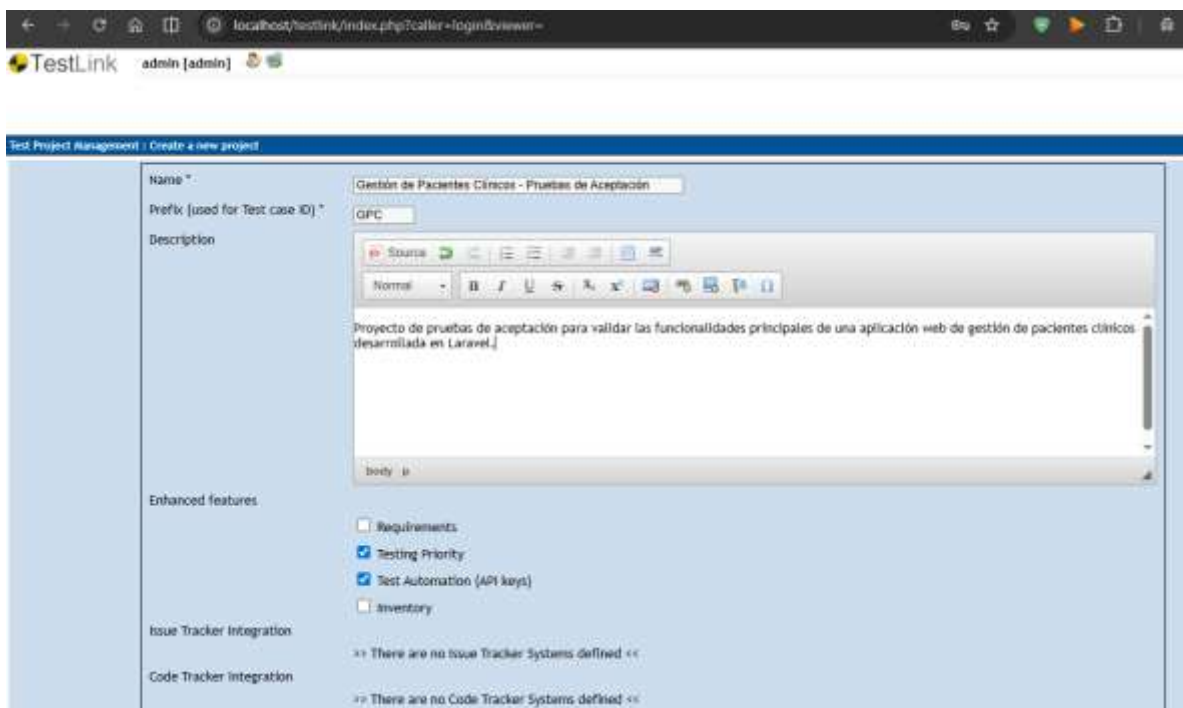
Estas pruebas permiten comprobar si la aplicación cumple con los requisitos esperados desde la perspectiva funcional, técnica y de aceptación. Además, ayudan a identificar posibles errores, validar permisos de acceso, proteger la información clínica sensible y documentar evidencias del comportamiento del sistema antes de una posible entrega al usuario final. La guía de la actividad solicita precisamente implementar pruebas de sistema y aceptación para el acceso a registros de pacientes clínicos.

Objetivo

Implementar un plan de pruebas de aceptación para la aplicación web de gestión de pacientes clínicos desarrollada en Laravel, con el fin de validar sus funcionalidades principales, controles de acceso, endpoints API, comportamiento de la interfaz, rendimiento y cumplimiento de los criterios esperados por el usuario final.

Diseño Casos de Prueba en TestLink

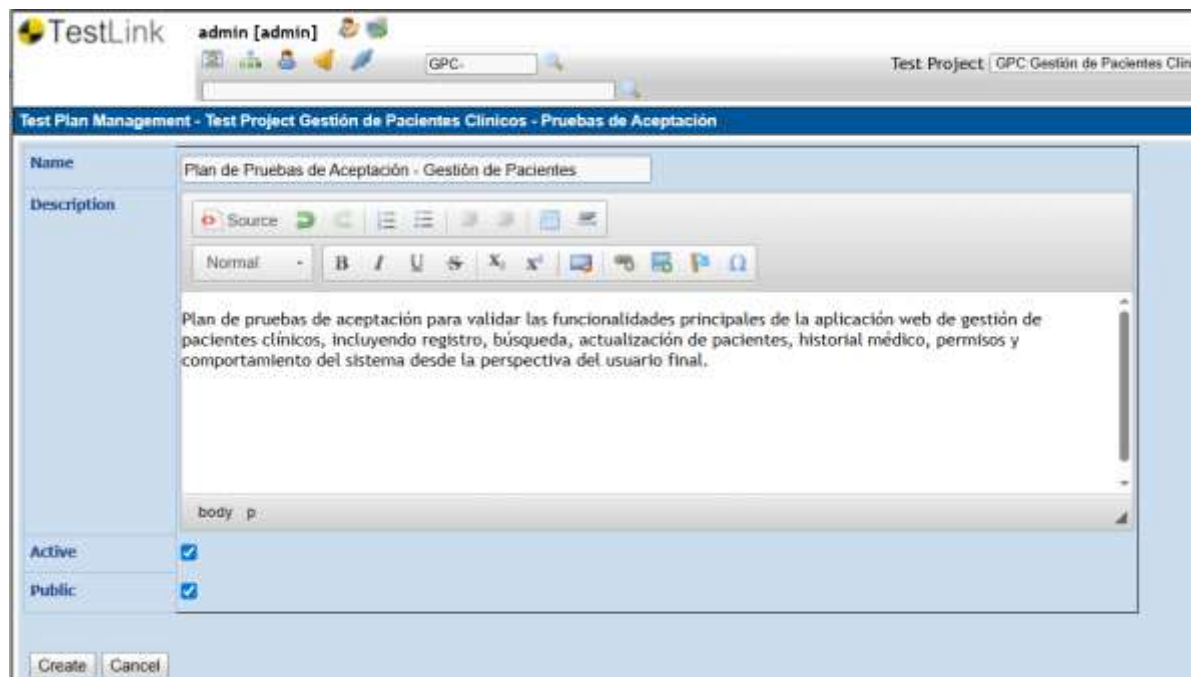
En esta primera parte se creó el proyecto de pruebas en TestLink para organizar los casos de aceptación de la aplicación web de gestión de pacientes clínicos. El proyecto permite centralizar las pruebas relacionadas con registro, búsqueda, actualización de pacientes e historial médico.



En la parte superior derecha se selecciona la opción de Test Plan Management



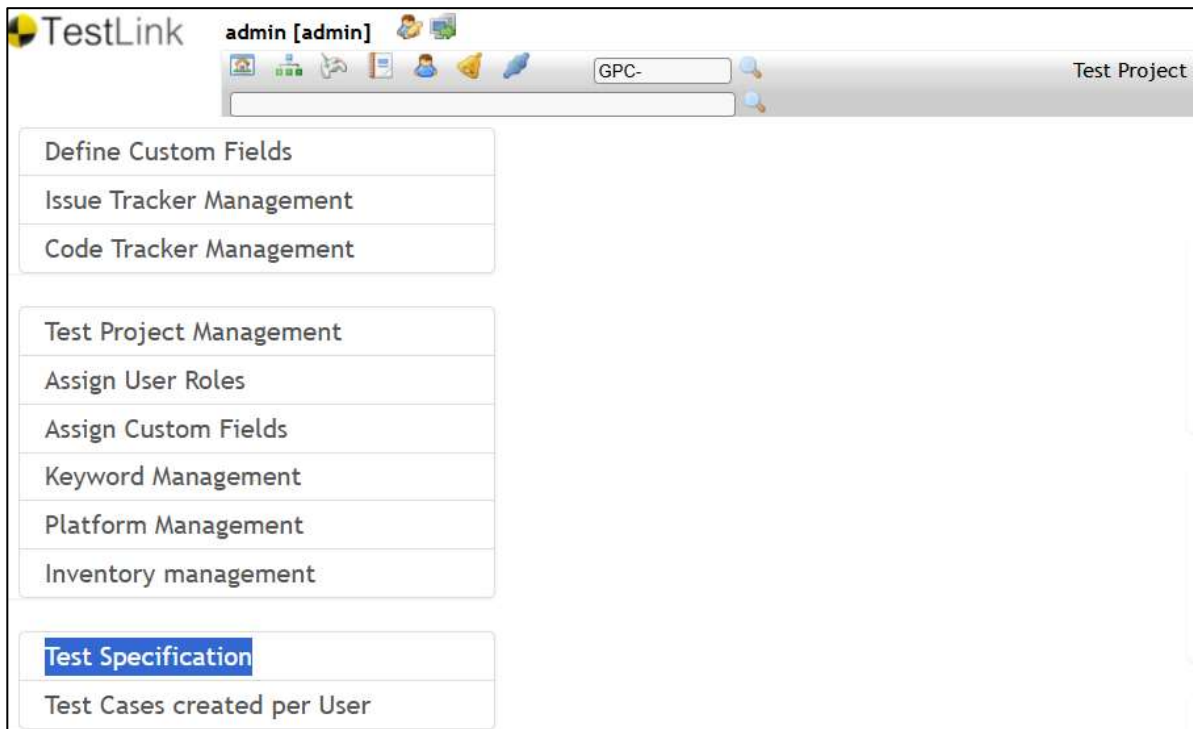
Se ingresó a la opción Test Plan Management para crear el plan de pruebas de aceptación. Este plan permite agrupar los casos de prueba que serán ejecutados posteriormente y registrar sus resultados.



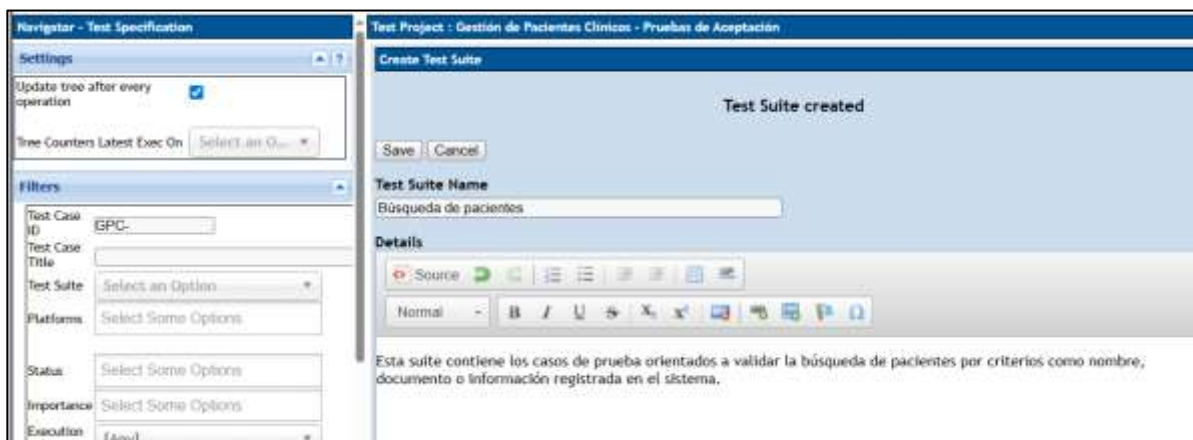
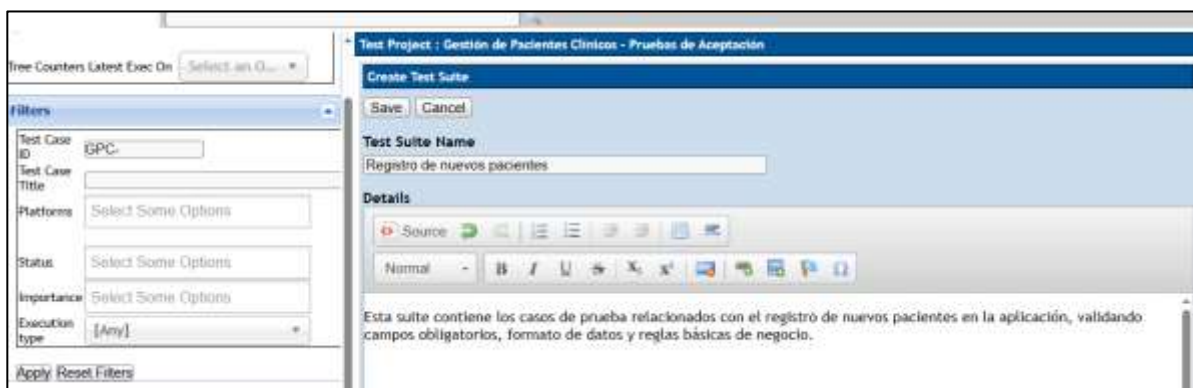
Se observa el plan de pruebas creado correctamente en TestLink. Este plan queda asociado al proyecto principal y será utilizado para vincular los casos de prueba diseñados en las diferentes suites.

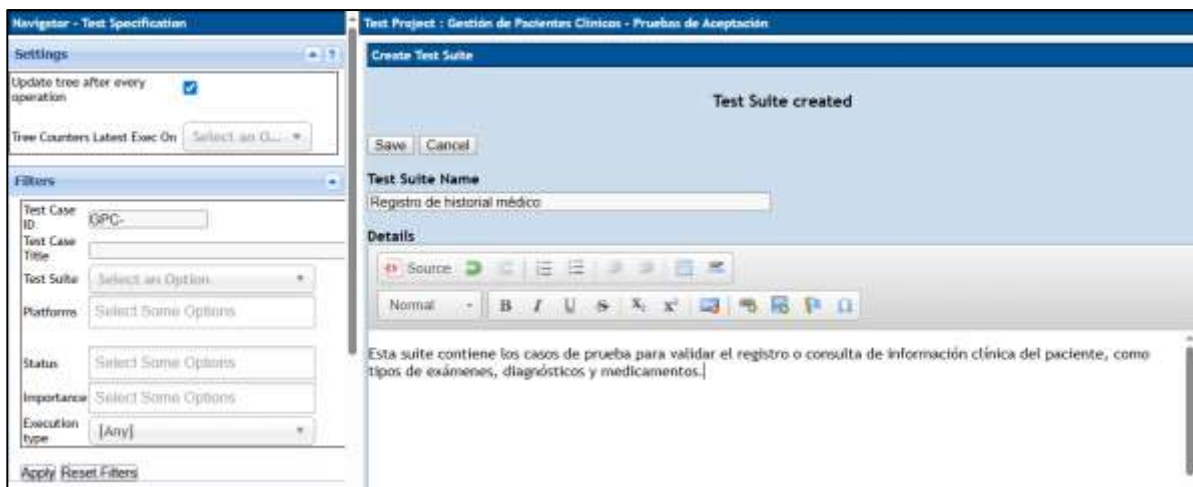
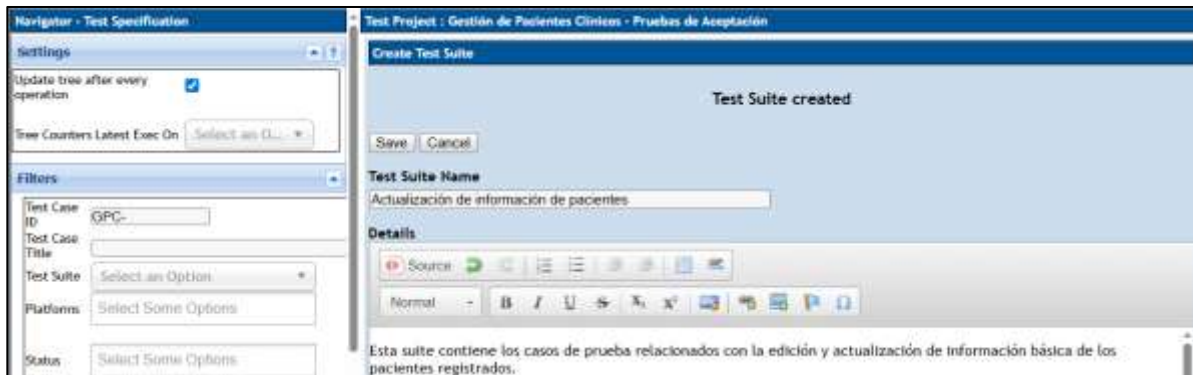


Después de crear el plan, se crea las carpetas donde estarán los casos de prueba. En el menú izquierdo, se busca y se entra a Test Specification.



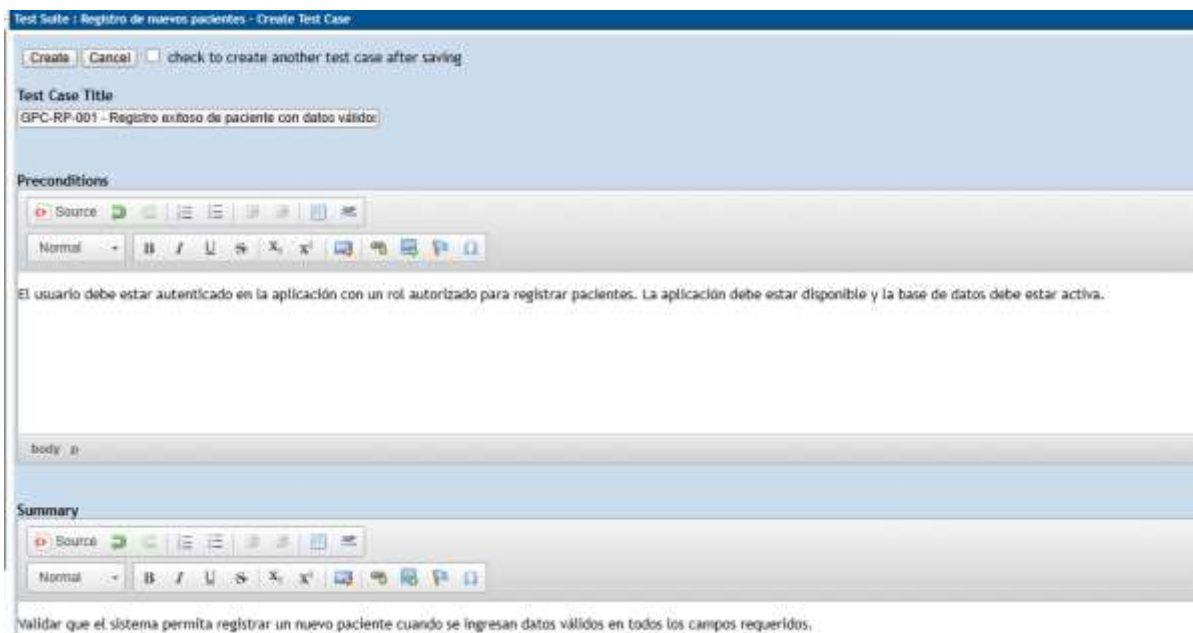
Se crearon cuatro suites principales de prueba: Registro de nuevos pacientes, Búsqueda de pacientes, Actualización de información de pacientes y Registro de historial médico.





Casos de Prueba para Registro de Nuevos pacientes

Dentro de cada suite se crearon casos de prueba específicos, definiendo título, precondiciones, resumen, importancia y tipo de ejecución. Para esta actividad, los casos se configuraron con ejecución manual, ya que TestLink se utilizó principalmente para documentar y gestionar las pruebas de aceptación.



Después de crear cada caso, se agregaron los pasos de ejecución junto con sus resultados esperados. Esta estructura permite que cualquier evaluador pueda seguir el flujo de prueba, ejecutar las acciones indicadas y comparar el comportamiento real del sistema con el resultado esperado.

GPC-1 : GPC-RP-001 - Registro exitoso de paciente con datos válidos - Versión 1			
Preconditions El usuario debe estar autenticado en la aplicación con un rol autorizado para registrar pacientes. La aplicación debe estar disponible y la base de datos debe estar activa.			
Summary Validar que el sistema permita registrar un nuevo paciente cuando se ingresan datos válidos en todos los campos requeridos.			
#	Step actions	Expected Results	Execution
1	Ingresar a la aplicación web de gestión de pacientes.	El sistema muestra la página principal o dashboard.	Manual
2	Realizar sesión con un usuario autorizado.	El sistema permite el acceso correctamente.	Manual
3	Ir a la opción de registro de paciente.	El sistema muestra el formulario de registro.	Manual
4	Ingresar documento, nombre completo, fecha de nacimiento, teléfono, correo, dirección y tipo de sangre válidos.	El sistema acepta los datos ingresados.	Manual
5	Guardar el registro.	El sistema registra al paciente y muestra mensaje de éxito.	Manual
6	Consultar al paciente registrado.	El paciente aparece en la lista o perfil correspondiente.	Manual

Test Case			
GPC-2 : GPC-RP-002 - Validación de campos obligatorios vacíos - Versión 1			
Preconditions El usuario debe estar autenticado con permisos para acceder al formulario de registro de pacientes.			
Summary Validar que el sistema no permita registrar un paciente cuando se dejan vacíos campos obligatorios como documento, nombre completo y fecha de nacimiento.			
#	Step actions	Expected Results	Execution
1	Ingresar al formulario de registro de pacientes.	El sistema muestra el formulario.	Manual
2	Dejar vacío el campo documento.	El sistema debe identificar que el campo es obligatorio.	Manual
3	Dejar vacío el campo nombre completo.	El sistema debe identificar que el campo es obligatorio.	Manual
4	Dejar vacío el campo fecha de nacimiento.	El sistema debe identificar que el campo es obligatorio.	Manual
5	Revisar los mensajes de validación.	El sistema muestra mensajes indicando los campos obligatorios faltantes.	Manual

Test Case			
GPC-3 : GPC-RP-003 - Validación de formato de datos del paciente - Versión 1			
Preconditions El usuario debe estar autenticado y ubicado en el formulario de registro de pacientes.			
Summary Validar que el sistema rechace datos con formato incorrecto, como correo electrónico inválido, fecha no válida o tipo de sangre no permitido.			
#	Step actions	Expected Results	Execution
1	Ingresar al formulario de registro de pacientes.	El formulario se muestra correctamente.	Manual
2	Ingresar un correo con formato incorrecto, por ejemplo: correo-invalido.	El sistema debe identificar el formato incorrecto.	Manual
3	Ingresar una fecha no válida.	El sistema debe rechazar la fecha.	Manual
4	Ingresar un tipo de sangre no permitido, por ejemplo: XYZ.	El sistema debe rechazar el valor ingresado.	Manual
5	Presionar guardar.	El sistema no debe registrar al paciente.	Manual
6	Revisar mensajes de error.	El sistema muestra mensajes claros sobre los campos inválidos.	Manual

Test Case		
GPC-4 : GPC-SP-004 - Validación de documento duplicado - Versión 1		
Preconditions Debe existir previamente un paciente registrado con un número de documento específico.		
Summary Validar que el sistema no permita registrar dos pacientes con el mismo número de documento.		
Step actions	Expected Results	Execution
1. Ingresar al formulario de registro de pacientes	El sistema muestra el formulario	Manual ✓ ✗
2. Ingresar un número de documento que ya existe en la base de datos	El sistema recibe el dato ingresado	Manual ✓ ✗
3. Completar los demás campos con datos válidos	Los campos se completan correctamente	Manual ✓ ✗
4. Presionar guardar	El sistema no debe registrar el paciente duplicado	Manual ✓ ✗
5. Revisar mensaje del sistema	El sistema indica que el documento ya se encuentra registrado	Manual ✓ ✗

Test Case		
GPC-5 : GPC-SP-005 - Validación de fecha de nacimiento futura - Versión 1		
Preconditions El usuario debe estar autenticado y ubicado en el formulario de registro de pacientes.		
Summary Validar que el sistema no permita registrar un paciente con una fecha de nacimiento posterior a la fecha actual.		
Step actions	Expected Results	Execution
1. Ingresar al formulario de registro de pacientes	El formulario se muestra correctamente	Manual ✓ ✗
2. Ingresar documento y nombre válidos	El sistema acepta los datos	Manual ✓ ✗
3. Ingresar una fecha de nacimiento futura	El sistema debe identificar que la fecha no es válida	Manual ✓ ✗
4. Completar los demás campos	Los campos restantes se diligencian	Manual ✓ ✗
5. Presionar guardar	El sistema no debe registrar al paciente	Manual ✓ ✗
6. Revisar mensaje de validación	El sistema muestra un mensaje indicando que la fecha de nacimiento no puede ser futura	Manual ✓ ✗

Casos de Prueba para Búsqueda de Pacientes

En esta suite se documentaron pruebas para validar la búsqueda de pacientes por documento, nombre y escenarios sin resultados. El objetivo es verificar que el sistema permita localizar pacientes registrados y que muestre mensajes adecuados cuando no existan coincidencias.

Test Case		
GPC-6 : GPC-SP-006 - Búsqueda exitosa de paciente por documento - Versión 1		
Preconditions El usuario debe estar autenticado en la aplicación. Debe existir al menos un paciente registrado en la base de datos con un número de documento válido.		
Summary Validar que el sistema permita buscar un paciente registrado utilizando su número de documento.		
Step actions	Expected Results	Execution
1. Ingresar a la aplicación web de gestión de pacientes	El sistema muestra la página de inicio o dashboard	Manual
2. Iniciar sesión con un usuario autorizado	El sistema permite el acceso correctamente	Manual
3. Ir a la sección de pacientes o búsqueda de pacientes	El sistema muestra la opción de búsqueda	Manual
4. Ingresar un número de documento existente	El sistema recibe el criterio de búsqueda	Manual
5. Presionar el botón buscar	El sistema realiza la búsqueda	Manual
6. Revisar los resultados obtenidos	El sistema muestra al paciente correspondiente al documento ingresado	Manual

Test Case

GPC-7 : GPC-SP-002 - Búsqueda exitosa de paciente por nombre - Versión 1

Preconditions

El usuario debe estar autenticado en la aplicación. Debe existir al menos un paciente registrado con nombre completo en la base de datos.

Summary

Validar que el sistema permita buscar un paciente registrado utilizando su nombre completo o parte del nombre.

#	Step actions	Expected Results	Execution
1	Ingresar a la aplicación web	El sistema carga correctamente	Manual
2	Iniciar sesión con un usuario autorizado	El sistema muestra el panel correspondiente al rol	Manual
3	Ir a la sección de búsqueda de pacientes	El sistema muestra el campo de búsqueda	Manual
4	Ingresar el nombre o parte del nombre de un paciente existente	El sistema acepta el texto ingresado	Manual
5	Presionar el botón buscar	El sistema procesa la búsqueda	Manual
6	Revisar los resultados	El sistema muestra uno o varios pacientes que coincidan con el nombre ingresado	Manual

Test Case



#	Step actions	Expected Results	Execution
1	Ingresar a la sección de búsqueda de pacientes	El sistema muestra el formulario o campo de búsqueda	Manual
2	Ingresar un documento o nombre que no exista en la base de datos	El sistema acepta el criterio ingresado	Manual
3	Presionar el botón buscar	El sistema ejecuta la búsqueda	Manual
4	Revisar el resultado mostrado	El sistema informa que no se encontraron pacientes con ese criterio	Manual
5	Verificar que no se muestre información incorrecta	El sistema no debe mostrar datos de otros pacientes	Manual

Test Case

 GPC-9 : GPC-SP-004 - Validación de búsqueda con campo vacío - Versión 1  

Preconditions 
El usuario debe estar autenticado y ubicado en la sección de búsqueda de pacientes.

Summary
Validar que el sistema controle adecuadamente una búsqueda cuando el usuario no ingresa ningún criterio.

#	Step actions	Expected Results	Execution
1	Ingresar a la sección de búsqueda de pacientes	El sistema muestra el campo de búsqueda	Manual
2	Dejar el campo de búsqueda vacío	El sistema mantiene el campo sin información	Manual
3	Presionar el botón buscar	El sistema valida la acción	Manual
4	Revisar el mensaje mostrado	El sistema solicita ingresar un criterio de búsqueda o muestra la lista general según la lógica definida	Manual
5	Verificar que no ocurra error del sistema	La aplicación no debe generar errores inesperados	Manual

Casos de Prueba para Actualización de Información de Pacientes

En esta suite se diseñaron pruebas relacionadas con la modificación de datos de pacientes. Se validan actualizaciones exitosas, campos obligatorios, formato de correo, cambios en datos de contacto y cancelación del proceso de edición.

Test Case		
 GPC-10 : GPC-AP-001 - Actualización externa de datos básicos del paciente - Versión 1		
Preconditions El usuario debe estar autenticado con permisos para actualizar pacientes. Debe existir al menos un paciente registrado en la base de datos.		
Summary Validar que el sistema permita actualizar correctamente los datos básicos de un paciente registrado.		
#	Step actions	Expected Results
1	Ingresar a la aplicación web de gestión de pacientes	El sistema muestra la página de inicio o dashboard.
2	Iniciar sesión con un usuario autorizado	El sistema permite el acceso correctamente.
3	Ir a la lista o búsqueda de pacientes	El sistema muestra los pacientes registrados.
4	Seleccionar un paciente existente	El sistema muestra la información del paciente.
5	Hacer clic en la opción editar o actualizar	El sistema muestra el formulario de edición.
6	Modificar datos como nombre, teléfono, correo o dirección	El sistema permite editar los campos.
7	Presionar el botón guardar o actualizar	El sistema guarda los cambios correctamente.
8	Consultar nuevamente al paciente	El sistema muestra la información actualizada.

Test Case			
GPC-11 : GPC-AP-002 - Validación de campos obligatorios al actualizar paciente - Versión 1			
Preconditions El usuario debe estar autenticado y debe existir un paciente registrado disponible para edición.			
Summary Validar que el sistema no permita actualizar la información de un paciente si se eliminan datos obligatorios.			
#	Step actions	Expected Results	Ejecución
1	Ingresar a la sección de pacientes	El sistema muestra la lista o búsqueda de pacientes.	Manual
2	Seleccionar un paciente existente	El sistema muestra el perfil o detalle del paciente.	Manual
3	Hacer clic en editar	El sistema abre el formulario de actualización.	Manual
4	Borrar el número de documento	El sistema debe identificar que el campo es obligatorio.	Manual
5	Borrar el nombre completo	El sistema debe identificar que el campo es obligatorio.	Manual
6	Borrar la fecha de nacimiento	El sistema debe identificar que el campo es obligatorio.	Manual
7	Presionar guardar	El sistema no debe actualizar el registro.	Manual
8	Revisar los mensajes de validación	El sistema muestra mensajes indicando los campos obligatorios faltantes.	Manual

Test Case



GPC-12 : GPC-AP-003 - Validación de formato de correo al actualizar paciente - Versión 1  

Preconditions 

El usuario debe estar autenticado con permisos de edición. Debe existir un paciente registrado.

Summary

Validar que el sistema rechace la actualización del paciente cuando se ingresa un correo electrónico con formato inválido.

#	Step actions	Expected Results	Ejecución
1	Ingresar a la sección de pacientes	El sistema muestra los pacientes registrados	Manual
2	Seleccionar un paciente existente	El sistema muestra la información del paciente	Manual
3	Hacer clic en editar	El sistema muestra el formulario de actualización	Manual
4	Cambiar el correo por un valor inválido, por ejemplo correo-invalido	El sistema recibe el dato ingresado	Manual
5	Presionar guardar	El sistema no debe actualizar el registro	Manual
6	Revisar el mensaje de error	El sistema muestra un mensaje indicando que el correo no tiene un formato válido	Manual

Test Case

GPC-13 : GPC-AP-004 - Actualización de teléfono y dirección del paciente - Versión 1

Preconditions

El usuario debe estar autenticado. Debe existir un paciente registrado con datos de contacto.

Summary

Validar que el sistema permita actualizar datos de contacto del paciente, como teléfono y dirección.

#	Step actions	Expected Results	Execution
1	Ingresar a la sección de pacientes	El sistema muestra la lista de pacientes	Manual
2	Seleccionar un paciente existente	El sistema muestra los datos del paciente	Manual
3	Hacer clic en editar	El sistema abre el formulario de actualización	Manual
4	Modificar el número de teléfono	El sistema permite cambiar el teléfono	Manual
5	Modificar la dirección	El sistema permite cambiar la dirección	Manual
6	Presionar guardar	El sistema actualiza la información	Manual
7	Consultar el perfil del paciente	El sistema muestra el nuevo teléfono y la nueva dirección	Manual

Test Case

GPC-14 : GPC-AP-005 - Cancelar actualización de información del paciente - Versión 1

Preconditions

El usuario debe estar autenticado y ubicado en el formulario de edición de un paciente registrado.

Summary

Validar que el sistema no modifique la información del paciente cuando el usuario cancela el proceso de actualización.

#	Step actions	Expected Results	Execution
1	Ingresar a la sección de pacientes	El sistema muestra los pacientes registrados	Manual
2	Seleccionar un paciente existente	El sistema muestra la información del paciente	Manual
3	Hacer clic en editar	El sistema muestra el formulario de actualización.	Manual
4	Modificar temporalmente uno o varios campos	El sistema permite editar los campos	Manual
5	Presionar cancelar o regresar sin guardar	El sistema sale del formulario sin guardar cambios.	Manual
6	Consultar nuevamente el paciente	El sistema conserva la información original	Manual

Casos de Prueba para Registro de Historial Médico

En esta suite se diseñaron casos para validar información clínica del paciente, como exámenes, diagnósticos, medicamentos, tratamientos y validaciones de campos vacíos. Estos casos permiten comprobar que la información médica se gestione de forma controlada y acorde con los permisos del sistema.

Test Case		
GPC-15 : GPC-IM-001 - Registro exitoso de historial médico - Versión 1		
Preconditions El usuario debe estar autenticado con rol doctor. Debe existir al menos un paciente registrado en la base de datos.		
Summary Validar que el sistema permita registrar correctamente información del historial médico de un paciente.		

Test Case

GPC-36 : GPC-191-002 - Registro de tipo de examen médico - Versión 1

Preconditions
 El usuario debe estar autenticado con rol doctor y debe existir un paciente registrado.

Summary
 Validar que el sistema permita registrar el tipo de examen médico asociado al paciente.

#	Step actions	Expected Results	Ejecución
1	Ingresar al panel del doctor	El sistema muestra las opciones permitidas para el rol doctor	Manual
2	Seleccionar un paciente existente	El sistema muestra el perfil del paciente	Manual
3	Ir a la sección de historial médico	El sistema muestra los campos clínicos disponibles	Manual
4	Ingresar un tipo de examen, por ejemplo: examen de sangre, radiografía o control general	El sistema acepta el dato ingresado	Manual
5	Guardar la información	El sistema registra el tipo de examen	Manual
6	Consultar el historial médico del paciente	El tipo de examen aparece asociado al paciente	Manual

Test Case

GPC-37 : GPC-191-003 - Registro de diagnóstico del paciente - Versión 1

Preconditions
 El usuario debe iniciar sesión con rol doctor. Debe existir un paciente disponible para consulta o edición.

Summary
 Validar que el sistema permita registrar un diagnóstico médico dentro del historial clínico del paciente.

#	Step actions	Expected Results	Ejecución
1	Ingresar a la aplicación como doctor	El sistema permite el acceso al panel médico	Manual
2	Buscar o seleccionar un paciente	El sistema muestra los datos del paciente	Manual
3	Ingresar a la sección de información clínica	El sistema muestra el campo de diagnóstico	Manual
4	Escribir un diagnóstico válido, por ejemplo: hipertensión arterial	El sistema permite ingresar el diagnóstico	Manual
5	Guardar los cambios	El sistema almacena el diagnóstico	Manual
6	Consultar nuevamente el perfil del paciente	El diagnóstico registrado se muestra correctamente al usuario doctor	Manual

Test Case

GPC-38 : GPC-191-004 - Registro de medicamentos o tratamientos - Versión 1

Preconditions
 El usuario debe estar autenticado con rol doctor. Debe existir un paciente registrado.

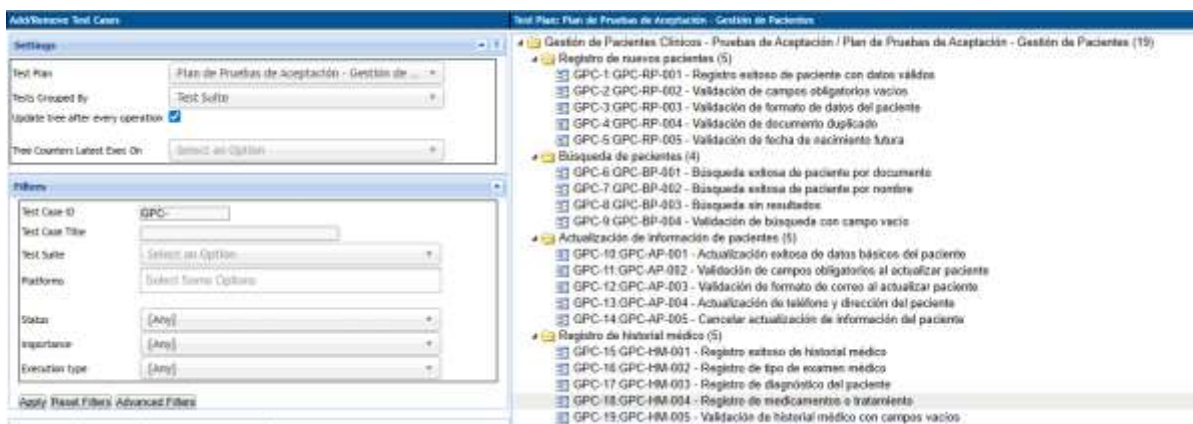
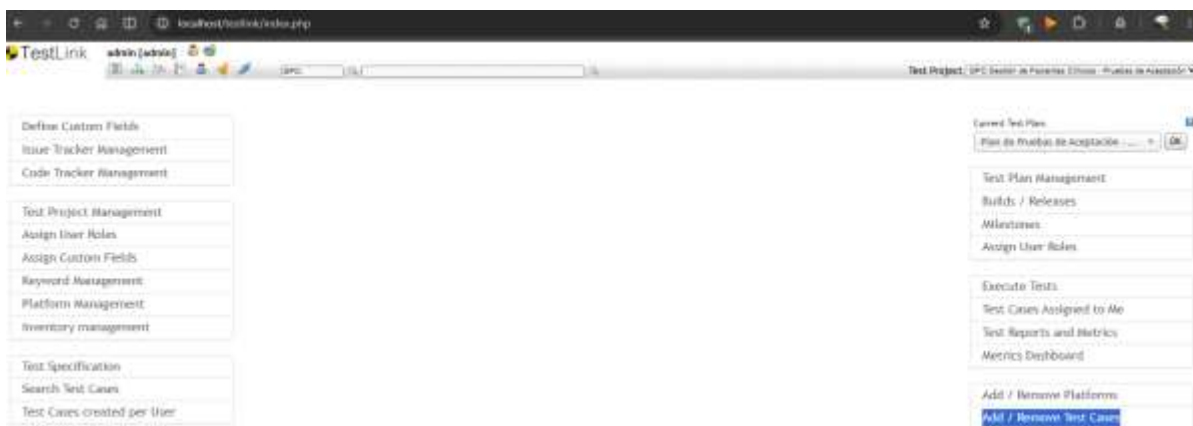
Summary
 Validar que el sistema permita registrar medicamentos, tratamiento o indicaciones médicas relacionadas con el paciente.

#	Step actions	Expected Results	Ejecución
1	Iniciar sesión como doctor	El sistema muestra el panel del doctor	Manual
2	Seleccionar un paciente registrado	El sistema muestra el perfil del paciente	Manual
3	Ir a la sección de historial médico o tratamiento	El sistema muestra el campo correspondiente	Manual
4	Registrar medicamentos o tratamiento indicado	El sistema permite ingresar la información	Manual
5	Guardar el registro	El sistema almacena el tratamiento correctamente	Manual
6	Consultar el perfil clínico del paciente	El medicamento o tratamiento registrado se visualiza correctamente	Manual

Test Case		
GPC-19 : GPC-HM-005 - Validación de historial médico con campos vacíos - Versión 1 Preconditions El usuario debe estar autenticado con rol doctor y ubicado en la sección de historial médico de un paciente. Summary GPC-HM-005 - Validación de historial médico con campos vacíos		
#	Step actions	Expected Results
1	Ingresar al panel del doctor	El sistema muestra las opciones disponibles
2	Seleccionar un paciente registrado	El sistema muestra la información del paciente
3	Ir a la sección de historial médico	El sistema muestra los campos clínicos
4	Dejar vacíos los campos obligatorios del historial médico	El sistema mantiene los campos sin información
5	Presionar guardar	El sistema no debe guardar información incompleta
6	Revisar mensajes de validación	El sistema muestra un mensaje indicando que se deben completar los campos requeridos

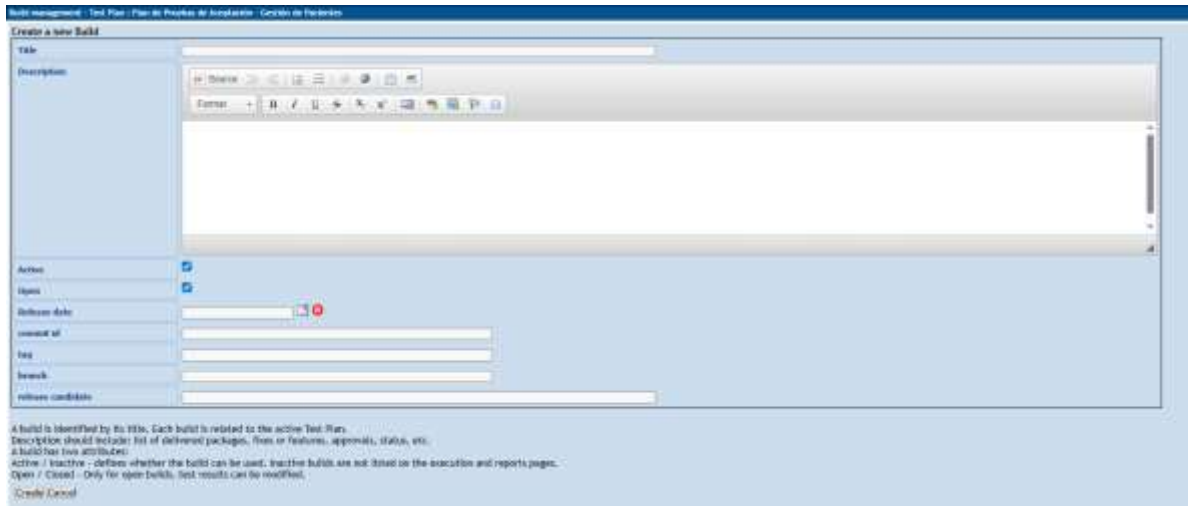
Asignar Casos al Plan de Pruebas

Después de crear los casos, se asignaron al plan de pruebas de aceptación. Esta asignación es necesaria para que los casos puedan ser ejecutados desde TestLink y se puedan registrar resultados como Passed, Failed o Blocked.



Ejecutar y Registrar Resultados en TestLink

Después de crear las suites y los casos de prueba, se realizó la asignación al plan de pruebas de aceptación y se creó un Build para permitir la ejecución manual de los casos en TestLink.



La ejecución de los casos permitió registrar el estado de cada prueba según el comportamiento observado en la aplicación. Los resultados utilizados fueron Passed, cuando el caso cumplió con el resultado esperado; Failed, cuando el comportamiento no coincidió con lo esperado; y Blocked, cuando la funcionalidad aún no se encontraba disponible o no podía ser validada completamente.

The screenshot shows the 'Latest execution' table in TestLink. The table has columns for Date, Build, Tested by, Status, Exec (min), Version, and Run mode. The first row shows a successful execution on 24/05/2026 at 23:31:50, tested by 'admin', with status 'Passed' and version '1'.

Latest execution (any build)						
Date: 24/05/2026 23:31:50 - Tested by: admin - Build: Build 1 - Aplicación Gestión de Pacientes Laravel - Status: Passed						
Latest execution (current build) - Build: Build 1 - Aplicación Gestión de Pacientes Laravel						
Date	Build	Tested by	Status	Exec (min)	Version	Run mode
24/05/2026 23:31:50	Build 1 - Aplicación Gestión de Pacientes Laravel	admin	Passed		1	

Esta ejecución permitió dejar evidencia del avance del plan de pruebas de aceptación y facilitó la identificación de funcionalidades implementadas, pendientes o que requieren ajustes.

Ejecutar Pruebas de Interfaz de Usuario con Selenium

Selenium permite automatizar acciones sobre el navegador, como abrir páginas, ingresar datos en formularios, hacer clic en botones y validar mensajes mostrados por la aplicación.

Preparación del Entorno Selenium

Para ejecutar las pruebas de interfaz de usuario solicitadas en la actividad, se preparó un entorno de automatización utilizando Selenium con Python.

La aplicación Laravel fue ejecutada localmente mediante el comando `php artisan serve`, quedando disponible en `http://127.0.0.1:8000`.

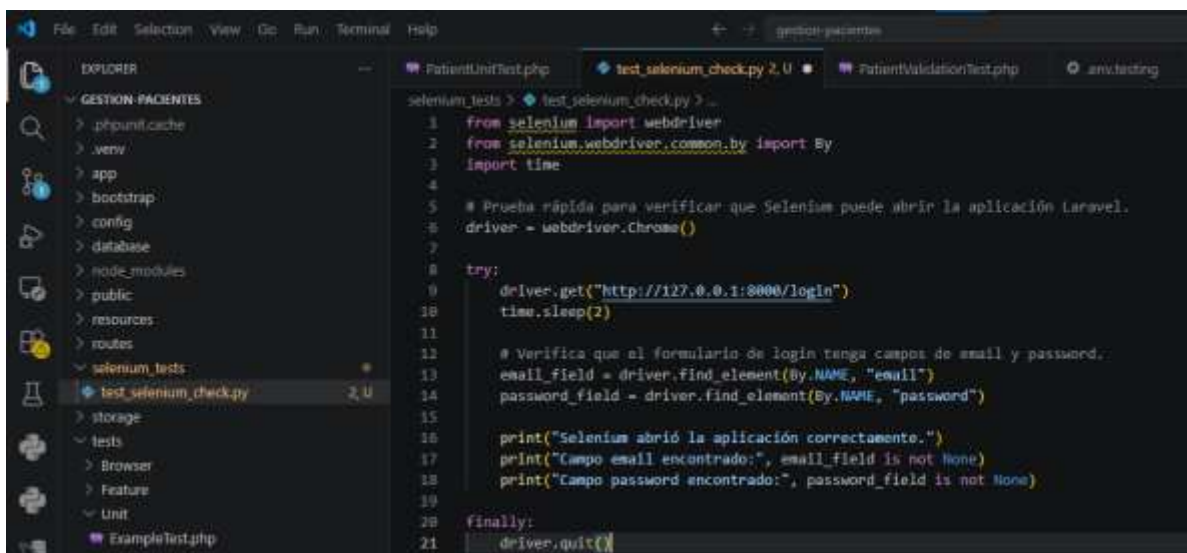
Posteriormente, se instaló la librería Selenium mediante pip.

```
python -c "import selenium; print(selenium.__version__)"  
4.44.0
```

Se creó la carpeta selenium_tests

Mode	LastWriteTime	Length	Name
----	-----	-----	----
d-----	24/05/2026 4:58 p. m.		selenium_tests

Y el archivo test_selenium_check.py



```
1 from selenium import webdriver  
2 from selenium.webdriver.common.by import By  
3 import time  
4  
5 # Prueba rápida para verificar que Selenium puede abrir la aplicación Laravel.  
6 driver = webdriver.Chrome()  
7  
8 try:  
9     driver.get("http://127.0.0.1:8000/login")  
10    time.sleep(2)  
11  
12    # Verifica que el formulario de login tenga campos de email y password.  
13    email_field = driver.find_element(By.NAME, "email")  
14    password_field = driver.find_element(By.NAME, "password")  
15  
16    print("Selenium abrió la aplicación correctamente.")  
17    print("Campo email encontrado:", email_field is not None)  
18    print("Campo password encontrado:", password_field is not None)  
19  
20 finally:  
21    driver.quit()
```

Como primera validación, se creó una prueba básica para abrir la página de inicio de sesión de la aplicación y verificar la existencia de los campos de correo electrónico y contraseña.

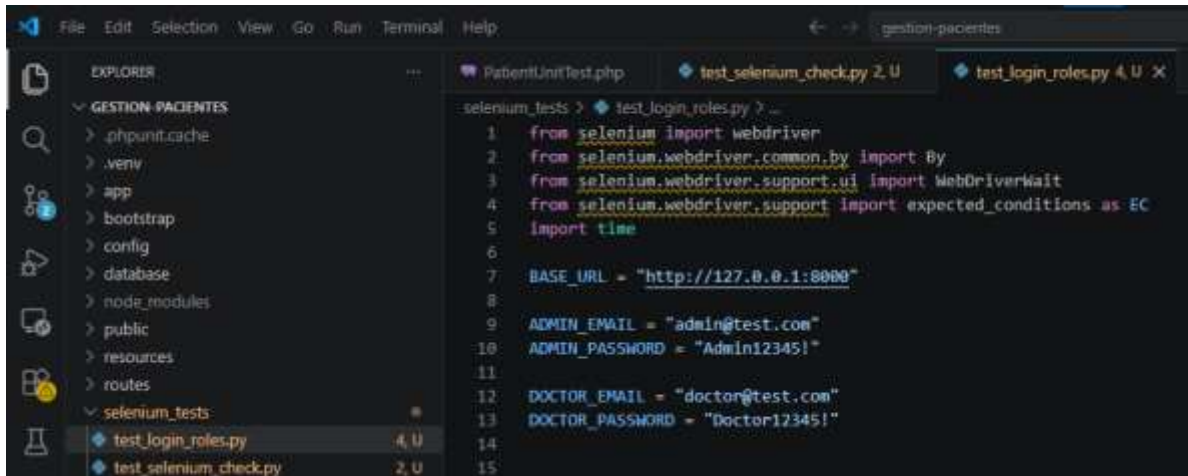
```
python selenium_tests/test_selenium_check.py
```

```
Selenium abrió la aplicación correctamente.  
Campo email encontrado: True  
Campo password encontrado: True
```

Esta prueba permitió confirmar que Selenium puede interactuar correctamente con la interfaz web del sistema.

Prueba Selenium de Login y Roles

Dentro de la carpeta: selenium_tests, se crea un archivo llamado: test_login_roles.py para validar el inicio de sesión y el acceso a los paneles protegidos de la aplicación. Esta prueba permite comprobar que los usuarios con rol admin y doctor puedan ingresar correctamente a las secciones correspondientes según sus permisos.



```
1 from selenium import webdriver
2 from selenium.webdriver.common.by import By
3 from selenium.webdriver.support.ui import WebDriverWait
4 from selenium.webdriver.support import expected_conditions as EC
5 import time
6
7 BASE_URL = "http://127.0.0.1:8000"
8
9 ADMIN_EMAIL = "admin@test.com"
10 ADMIN_PASSWORD = "Admin12345!"
11
12 DOCTOR_EMAIL = "doctor@test.com"
13 DOCTOR_PASSWORD = "Doctor12345!"
14
15
```

La ejecución del script test_login_roles.py abre automáticamente el navegador, inicia sesión con los usuarios de prueba y valida los accesos permitidos y denegados.



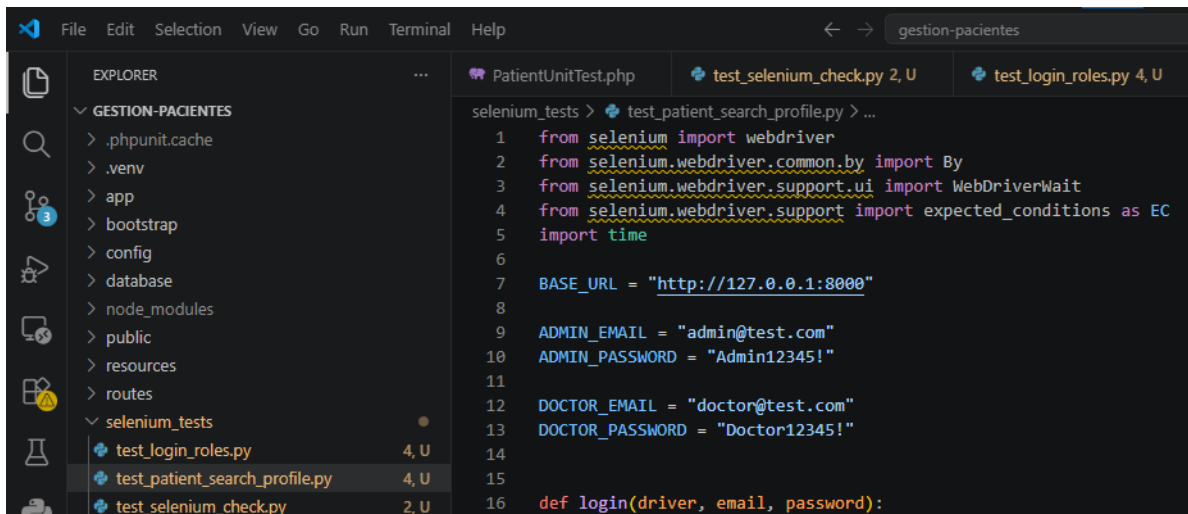
Todas las validaciones se cumplieron y la terminal muestra que las pruebas finalizaron correctamente.

```
python selenium_tests/test_login_roles.py
```

```
Prueba admin: acceso al panel administrador correcto.
Prueba doctor: acceso al panel doctor correcto.
Prueba permisos: admin bloqueado correctamente al intentar entrar al panel doctor.
Prueba permisos: doctor bloqueado correctamente al intentar entrar al panel admin.
Todas las pruebas Selenium de login y roles finalizaron correctamente.
```

Prueba Selenium para Búsqueda/Consulta de Pacientes

Dentro de la carpeta `selenium_tests`, se crea un archivo llamado: `test_patient_search_profile.py` para validar la consulta de pacientes desde la interfaz web de la aplicación. Esta prueba permite comprobar que los usuarios autenticados puedan acceder a la información de pacientes según su rol.



```
File Edit Selection View Go Run Terminal Help
EXPLORER
GESTION-PACIENTES
  .phpunit.cache
  .venv
  app
  bootstrap
  config
  database
  node_modules
  public
  resources
  routes
  selenium_tests
    test_login_roles.py 4, U
    test_patient_search_profile.py 4, U
    test_selenium_check.py 2, U
PatientUnitTest.php
test_selenium_check.py 2, U
test_login_roles.py 4, U
selenium_tests > test_patient_search_profile.py > ...
1 from selenium import webdriver
2 from selenium.webdriver.common.by import By
3 from selenium.webdriver.support.ui import WebDriverWait
4 from selenium.webdriver.support import expected_conditions as EC
5 import time
6
7 BASE_URL = "http://127.0.0.1:8000"
8
9 ADMIN_EMAIL = "admin@test.com"
10 ADMIN_PASSWORD = "Admin12345!"
11
12 DOCTOR_EMAIL = "doctor@test.com"
13 DOCTOR_PASSWORD = "Doctor12345!"
14
15
16 def login(driver, email, password):
```

Al ejecutar el script `test_patient_search_profile.py`, se abre el navegador automáticamente, inicia sesión con los usuarios de prueba y valida la consulta de pacientes según los permisos de cada rol.

```
python selenium_tests/test_patient_search_profile.py
```



Dashboard

Panel Admin

Administrador ▾

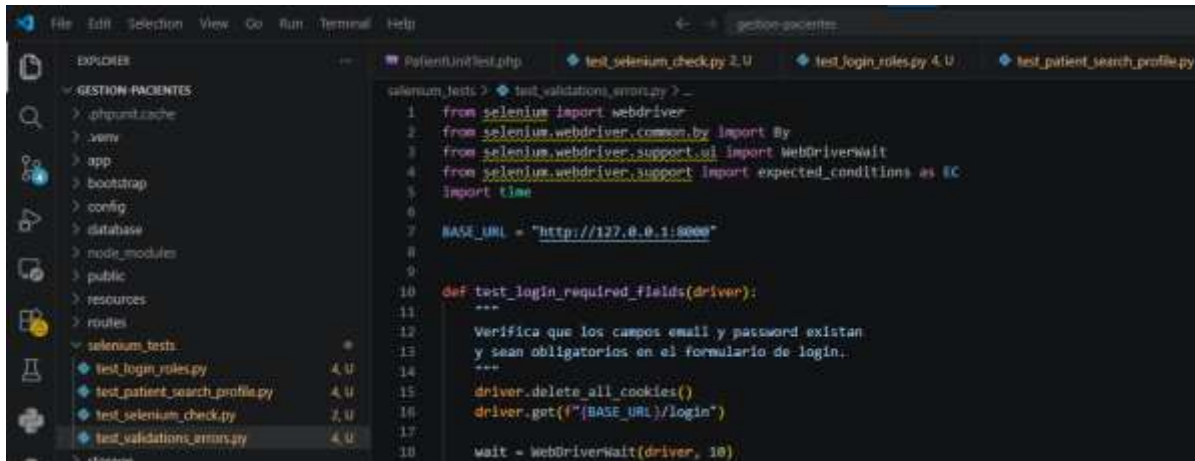
Perfil del Paciente

María Fernanda López

```
Admin: se visualiza la sección o lista de pacientes.
Admin: acceso al perfil de paciente con información limitada correcto.
Doctor: se visualiza la sección o lista de pacientes.
Doctor: acceso al perfil clínico del paciente correcto.
Todas las pruebas Selenium de búsqueda/consulta de pacientes finalizaron correctamente.
```

Prueba Selenium de Validaciones y Mensajes de Error

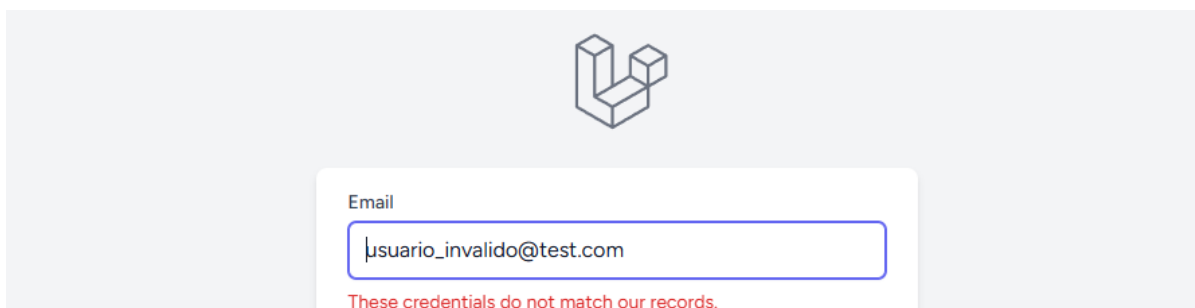
Dentro de la carpeta `selenium_tests`, se crea un archivo llamado: `test_validations_errors.py` para validar el comportamiento del sistema frente a errores de entrada y accesos no autorizados. Esta prueba se enfocó en el formulario de inicio de sesión y en rutas protegidas de la aplicación.



```
1 from selenium import webdriver
2 from selenium.webdriver.common.by import By
3 from selenium.webdriver.support.ui import WebDriverWait
4 from selenium.webdriver.support import expected_conditions as EC
5 import time
6
7 BASE_URL = "http://127.0.0.1:8000"
8
9
10 def test_login_required_fields(driver):
11     """
12     Verifica que los campos email y password existan
13     y sean obligatorios en el formulario de login.
14     """
15     driver.delete_all_cookies()
16     driver.get(f"{BASE_URL}/login")
17
18     wait = WebDriverWait(driver, 10)
```

El script `test_validations_errors.py` abre el navegador, ejecuta los escenarios definidos y muestra en la terminal que todas las validaciones finalizaron correctamente.

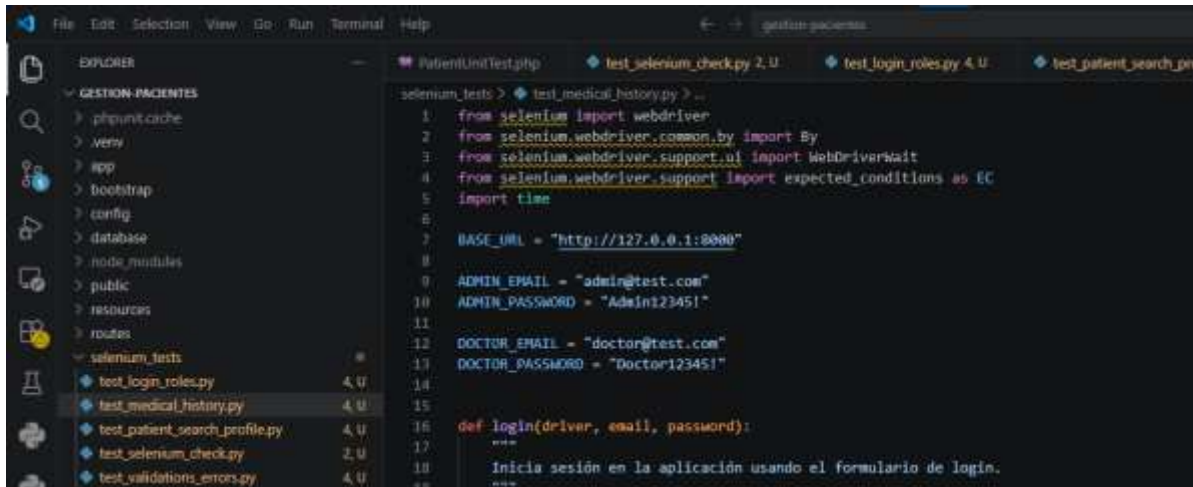
```
python selenium_tests/test_validations_errors.py
```



```
Validación login: campos email y password son obligatorios.
Validación login: credenciales incorrectas rechazadas correctamente.
Validación permisos: usuario no autenticado redirigido al login al intentar entrar a /admin.
Validación permisos: usuario no autenticado redirigido al login al intentar entrar a /patients/1.
Todas las pruebas Selenium de validaciones y mensajes de error finalizaron correctamente.
```

Prueba Selenium para Historial Médico

Dentro de la carpeta selenium_tests, se crea un archivo llamado: test_medical_history.py para validar la visualización de información clínica del paciente según el rol del usuario autenticado.

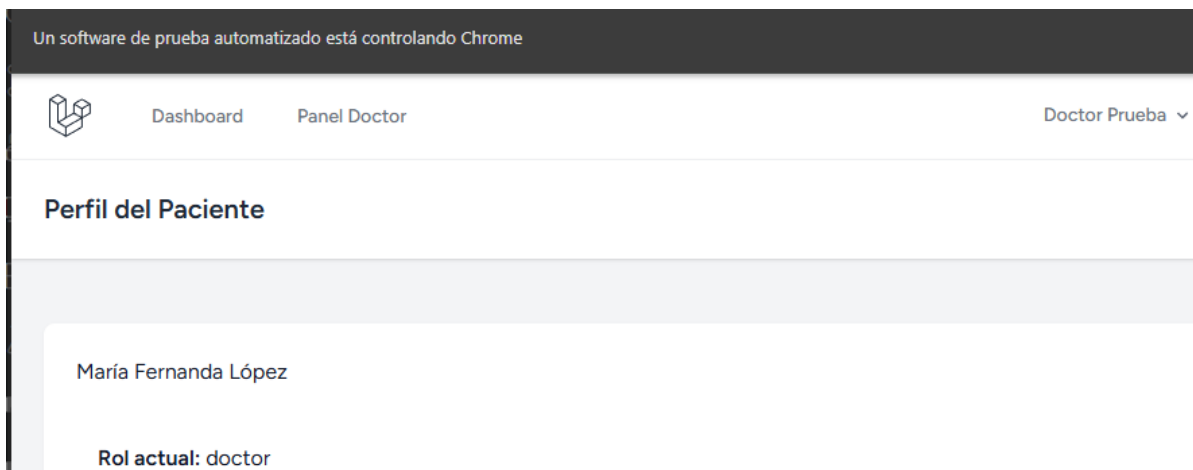


```
EXPLORER
└─ GESTION-PACIENTES
   └─ selenium_tests
      ├── test_login_roles.py 4 U
      ├── test_medical_history.py 4 U
      ├── test_patient_search_profile.py 4 U
      ├── test_selenium_check.py 2 U
      └── test_validations_errors.py 4 U

selenium_tests > test_medical_history.py > ...
1 from selenium import webdriver
2 from selenium.webdriver.common.by import By
3 from selenium.webdriver.support.ui import WebDriverWait
4 from selenium.webdriver.support import expected_conditions as EC
5 import time
6
7 BASE_URL = "http://127.0.0.1:8000"
8
9 ADMIN_EMAIL = "admin@test.com"
10 ADMIN_PASSWORD = "Admin12345!"
11
12 DOCTOR_EMAIL = "doctor@test.com"
13 DOCTOR_PASSWORD = "Doctor12345!"
14
15
16 def login(driver, email, password):
17     """
18     Inicia sesión en la aplicación usando el formulario de login.
19     """
```

El script test_medical_history.py abre el navegador automáticamente, inicia sesión con los usuarios de prueba y valida que la información clínica sea visible únicamente para el rol autorizado.

```
python selenium_tests/test_medical_history.py
```



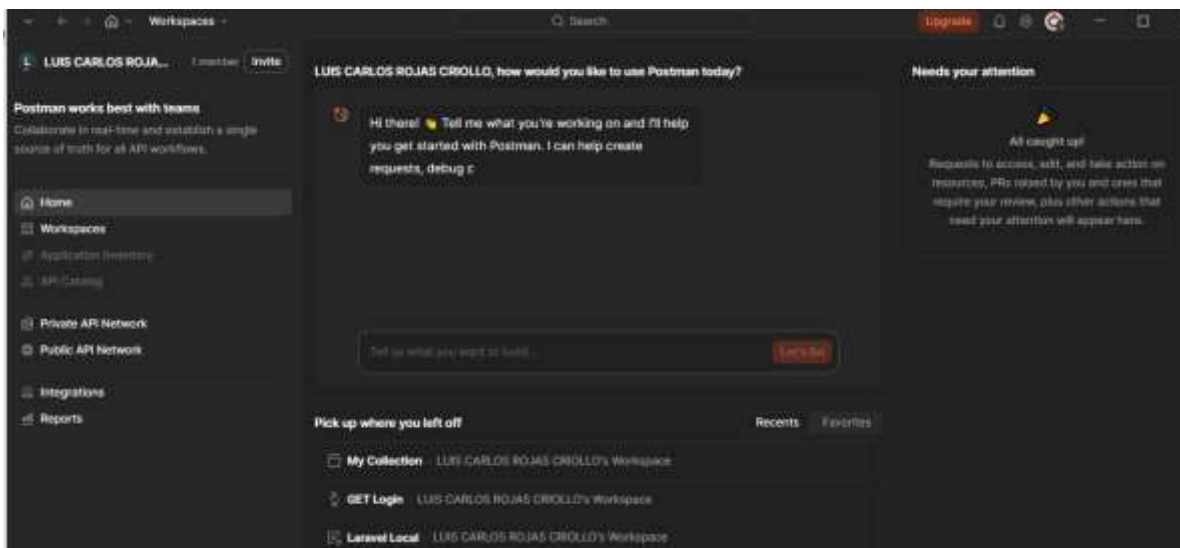
La prueba permitió comprobar que el usuario con rol doctor puede acceder al perfil clínico del paciente y visualizar información médica como diagnóstico, tratamiento o notas médicas. También se validó que el usuario con rol admin no pueda visualizar información clínica sensible, manteniendo la separación de permisos definida en la aplicación.

```
Historial médico: el doctor puede visualizar información clínica del paciente.
Historial médico: el admin no visualiza información clínica sensible.
Todas las pruebas Selenium de historial médico finalizaron correctamente.
```

Pruebas de API con Postman

Preparación de Postman para Pruebas de API

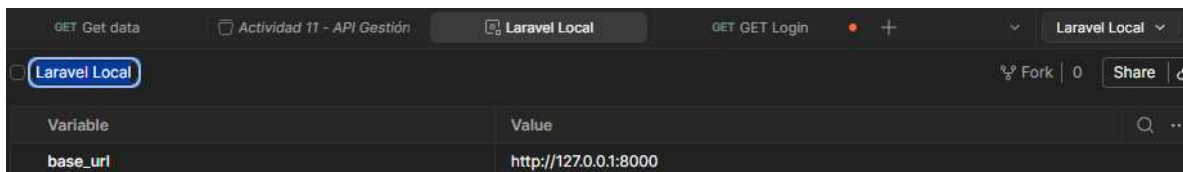
Para ejecutar las pruebas de API solicitadas en la actividad, se instaló la aplicación de escritorio de Postman en Windows. Esta herramienta permite crear colecciones de solicitudes HTTP, enviar peticiones a la aplicación y validar respuestas del servidor.



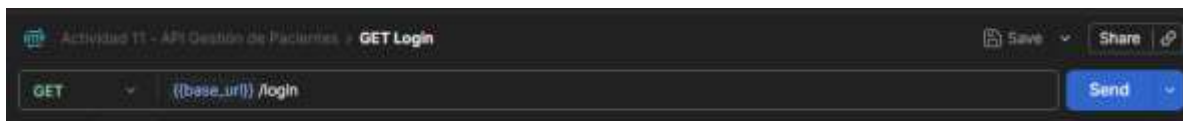
Posteriormente, se creó una colección llamada Actividad 11 - API Gestión de Pacientes, destinada a organizar las pruebas relacionadas con endpoints de pacientes, historial médico, doctores, seguridad y manejo de errores.



También se configuró un entorno llamado Laravel Local, con la variable base_url apuntando a http://127.0.0.1:8000, correspondiente a la aplicación Laravel ejecutada localmente mediante php artisan serve.



Como primera validación, se creó una solicitud GET {{base_url}}/login, la cual permitió comprobar que Postman podía comunicarse correctamente con la aplicación local.



Al orprimir Send, aparece 200 OK y en el cuerpo de la respuesta el HTML del login o contenido relacionado con el formulario.

```

1 <!DOCTYPE html>
2 <html lang="en">
3
4 <head>
5     <meta charset="utf-8">
6     <meta name="viewport" content="width=device-width, initial-scale=1">
7     <meta name="csrf-token" content="K66CasaJWuE30U0WXoAFtnK7qIWNmWnhZ0sUc08">
8
9     <title>Laravel</title>
10
11 <!-- Fonts -->

```

Creación de Endpoints API en Laravel

Para desarrollar las pruebas de API solicitadas en la actividad, se crearon endpoints básicos en Laravel relacionados con pacientes, doctores e historial médico. Estos endpoints permiten consultar pacientes, registrar nuevos pacientes, actualizar información, consultar doctores y gestionar información clínica básica asociada a un paciente.

Método	Endpoint	Descripción
GET	/api/patients	Lista los pacientes registrados
GET	/api/patients/{id}	Consulta un paciente específico
POST	/api/patients	Registra un nuevo paciente
PUT	/api/patients/{id}	Actualiza información de un paciente
GET	/api/doctors	Lista los usuarios con rol doctor
GET	/api/patients/{id}/medical-history	Consulta historial médico básico
POST	/api/patients/{id}/medical-history	Actualiza información clínica del paciente

Además, se implementó un middleware llamado CheckApiKey, encargado de validar que las solicitudes incluyan el encabezado X-API-KEY. Esta medida permite probar escenarios de seguridad y manejo de errores desde Postman.

```

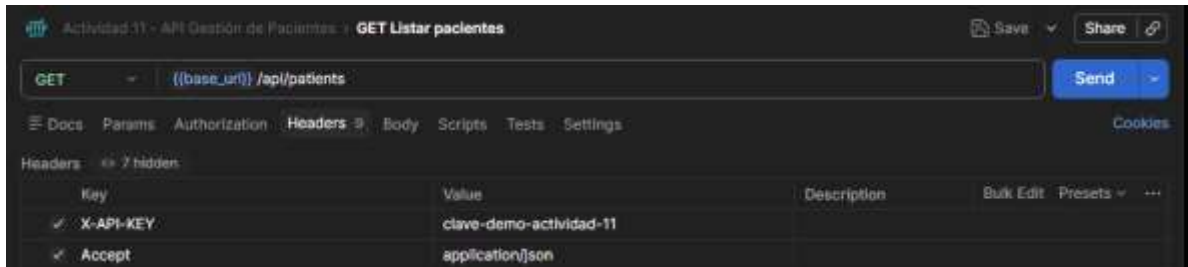
1 <?php
2
3 namespace App\Http\Middleware;
4
5 use Closure;
6 use Illuminate\Http\Request;
7 use Symfony\Component\HttpFoundation\Response;
8
9 class CheckApiKey
10 {
11     /**
12      * Middleware básico para proteger los endpoints API.
13      * Valida que la petición incluya el encabezado X-API-KEY.
14      */
15     public function handle(Request $request, Closure $next): Response
16     {
17         $validKey = env('API_KEY', 'clave-demo-actividad-11');
18
19         if ($request->header('X-API-KEY') !== $validKey) {
20             return response()->json([
21                 'message' => 'No autorizado. API Key inválida o ausente.'
22             ], 401);
23         }
24
25         return $next($request);
26     }
27 }

```

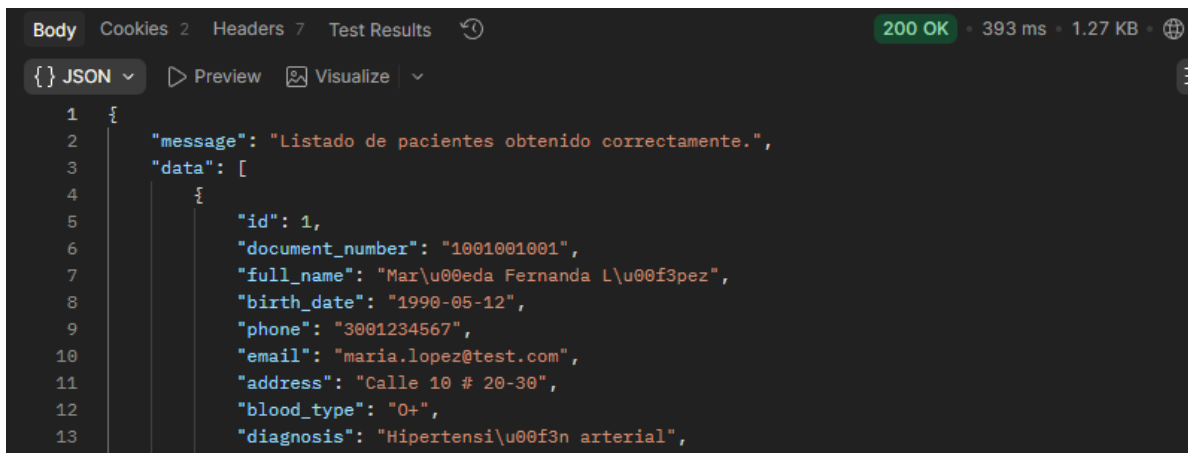

Endpoints de Pacientes en Postman

Request 1: Listar pacientes

Se configura una petición GET al endpoint `/api/patients`, incluyendo el encabezado X-API-KEY para autorizar la consulta desde Postman.

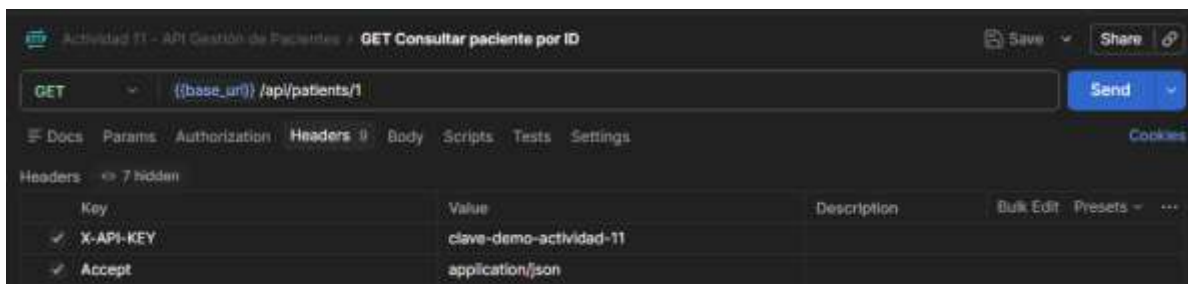


La API responde con estado 200 OK y retorna el listado de pacientes registrados en la base de datos.

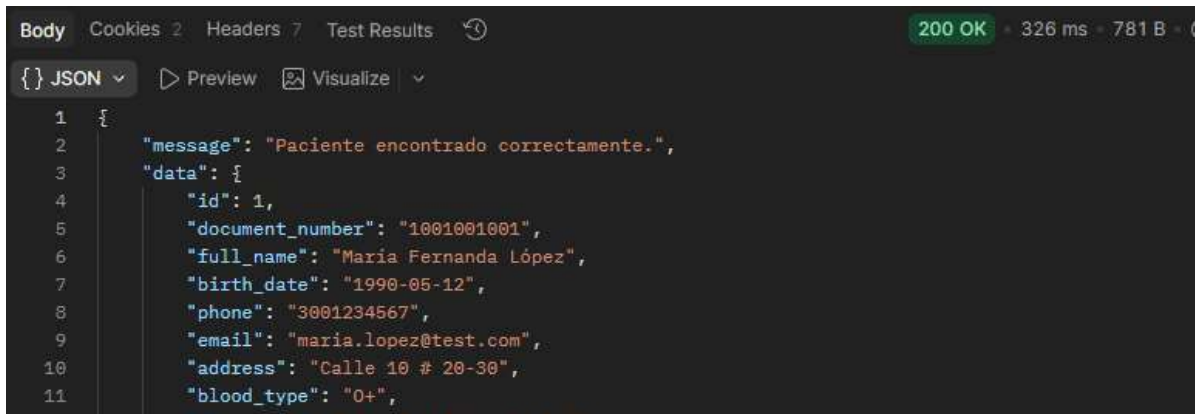


Request 2: Consultar paciente por ID

Se realiza una petición GET al endpoint `/api/patients/{id}` para consultar la información de un paciente específico.



El sistema responde con 200 OK y muestra los datos del paciente solicitado, confirmando que la consulta por identificador funciona correctamente.

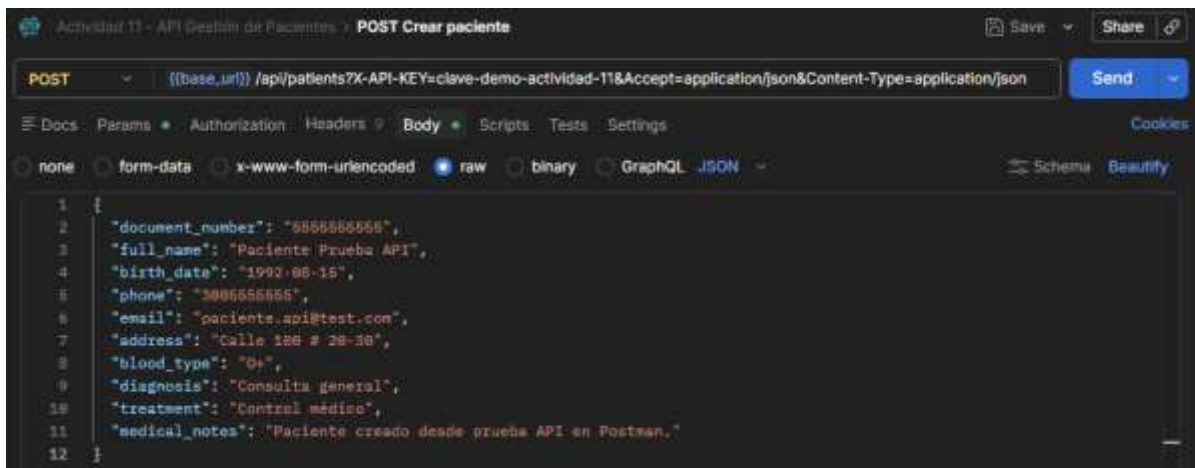


The screenshot shows a REST client interface with the 'Body' tab selected. The response is a JSON object with a status of 200 OK, 326 ms, and 781 B. The JSON body is as follows:

```
1 {
2   "message": "Paciente encontrado correctamente.",
3   "data": {
4     "id": 1,
5     "document_number": "1001001001",
6     "full_name": "Maria Fernanda López",
7     "birth_date": "1990-05-12",
8     "phone": "3001234567",
9     "email": "maria.lopez@test.com",
10    "address": "Calle 10 # 20-30",
11    "blood_type": "O+",
12  }
```

Request 3: Crear un nuevo paciente

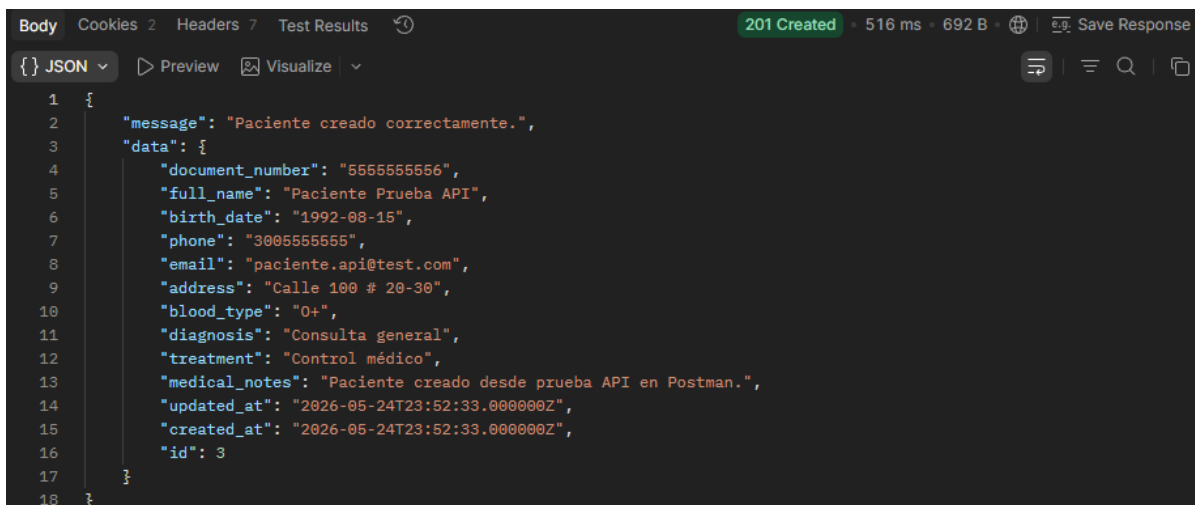
Se configura una petición POST al endpoint `/api/patients`, enviando en el cuerpo de la solicitud los datos básicos y clínicos del paciente en formato JSON.



The screenshot shows a REST client interface with the 'Body' tab selected. The request is a POST to the endpoint `/api/patients` with a status of 201 Created, 516 ms, and 692 B. The JSON body is as follows:

```
1 {
2   "document_number": "5555555555",
3   "full_name": "Paciente Prueba API",
4   "birth_date": "1992-08-15",
5   "phone": "3005555555",
6   "email": "paciente.api@test.com",
7   "address": "Calle 100 # 20-30",
8   "blood_type": "O+",
9   "diagnosis": "Consulta general",
10  "treatment": "Control médico",
11  "medical_notes": "Paciente creado desde prueba API en Postman.",
12 }
```

La API responde con 201 Created, indicando que el paciente fue registrado correctamente en la base de datos.

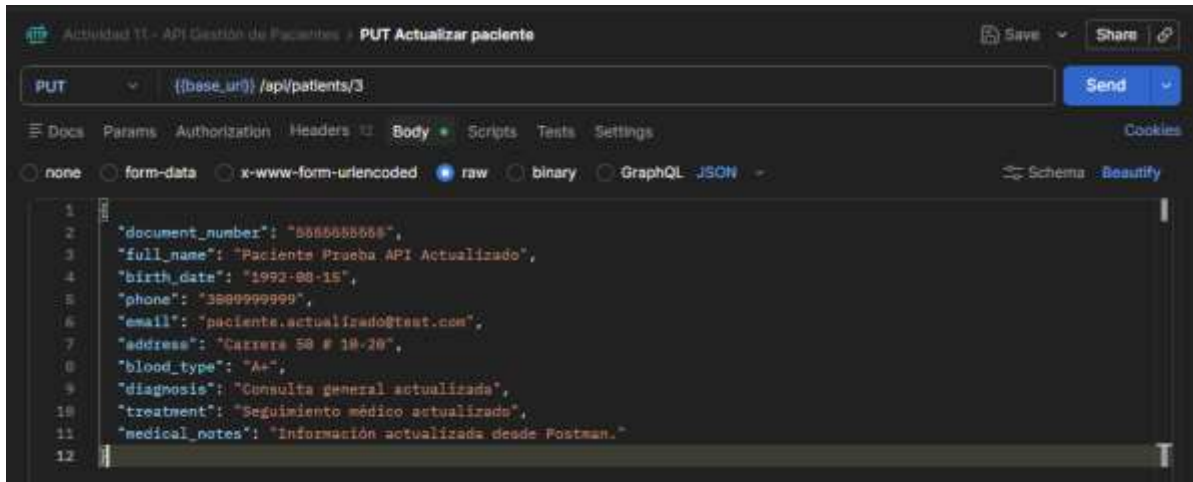


The screenshot shows a REST client interface with the 'Body' tab selected. The response is a JSON object with a status of 201 Created, 516 ms, and 692 B. The JSON body is as follows:

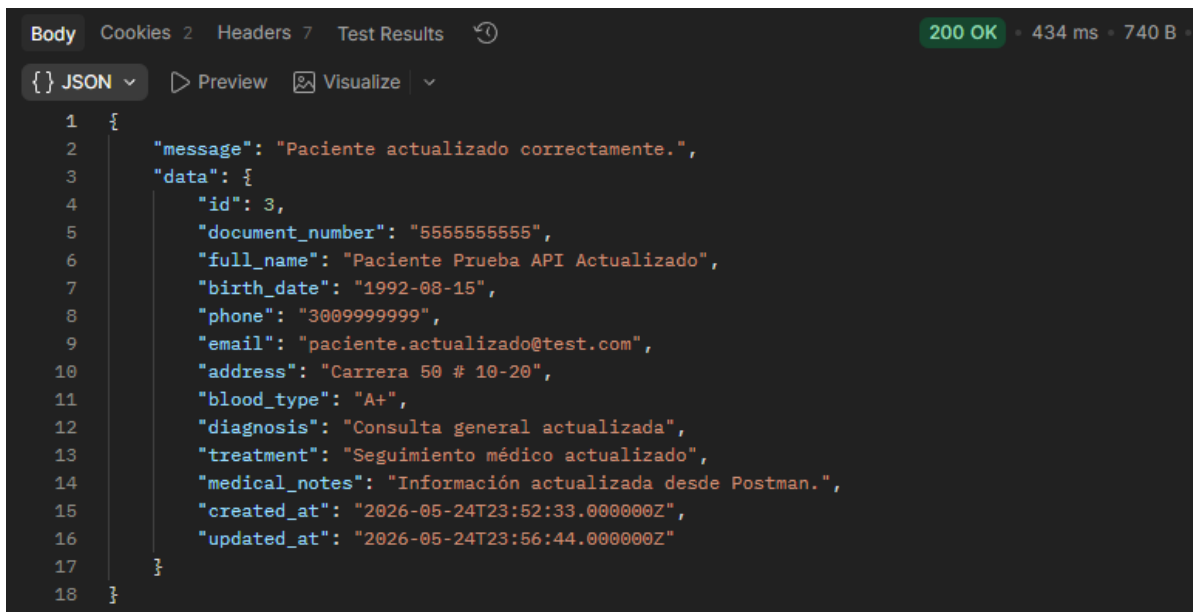
```
1 {
2   "message": "Paciente creado correctamente.",
3   "data": {
4     "document_number": "5555555556",
5     "full_name": "Paciente Prueba API",
6     "birth_date": "1992-08-15",
7     "phone": "3005555555",
8     "email": "paciente.api@test.com",
9     "address": "Calle 100 # 20-30",
10    "blood_type": "O+",
11    "diagnosis": "Consulta general",
12    "treatment": "Control médico",
13    "medical_notes": "Paciente creado desde prueba API en Postman.",
14    "updated_at": "2026-05-24T23:52:33.000000Z",
15    "created_at": "2026-05-24T23:52:33.000000Z",
16    "id": 3
17  }
18 }
```

Request 4: Actualizar paciente

Se realiza una petición PUT al endpoint `/api/patients/{id}`, enviando los datos actualizados del paciente en formato JSON.

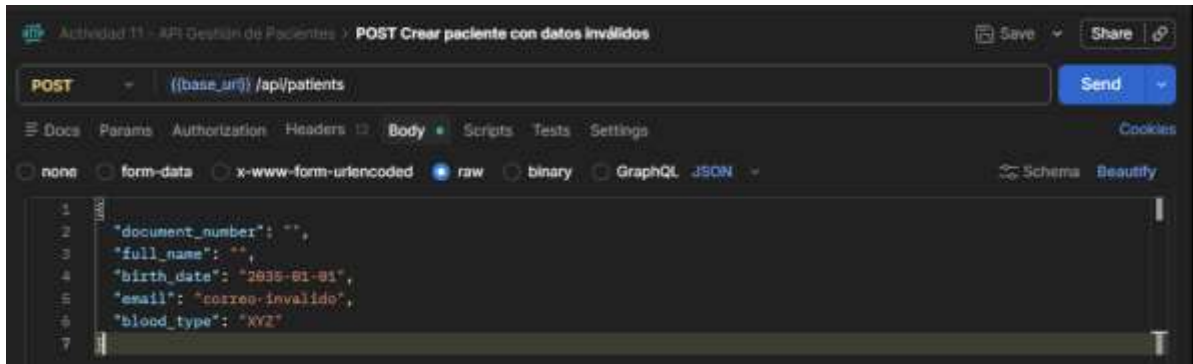


El sistema responde con 200 OK y confirma que la información del paciente fue actualizada correctamente.

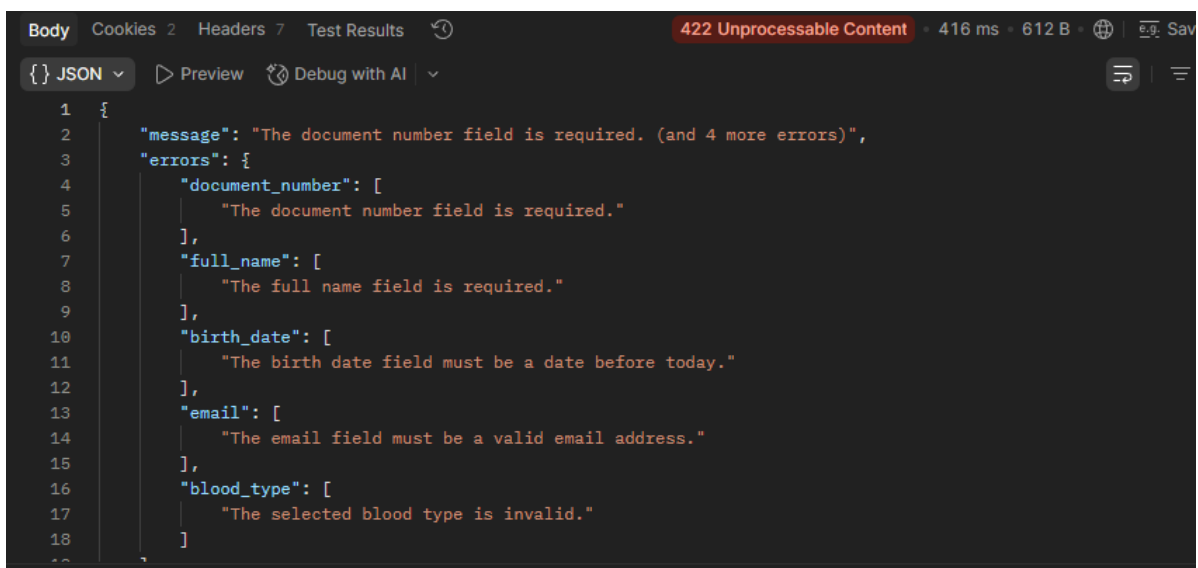


Request 5: Validar error por datos inválidos

Se envía una petición POST con campos vacíos o datos inválidos, como correo incorrecto, fecha no permitida o tipo de sangre no válido.



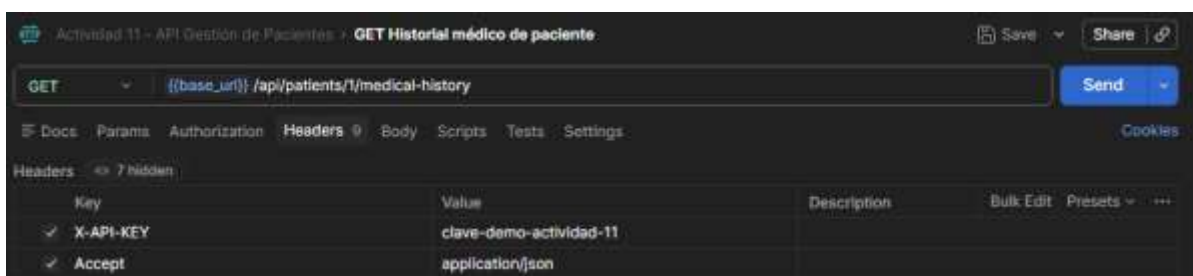
La API responde con 422 Unprocessable Entity, mostrando los errores de validación correspondientes a los campos incorrectos.



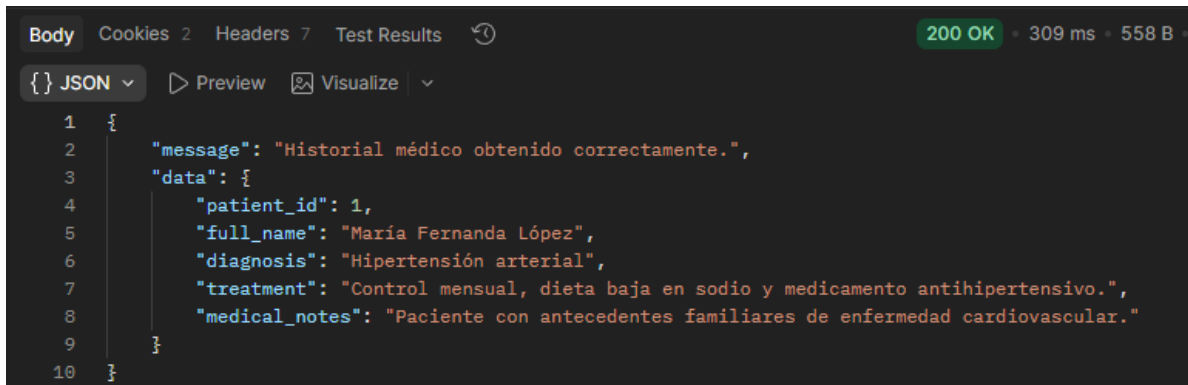
Endpoints de Historial Médico en Postman

Request 1: Consultar historial médico de un paciente

Se configura una petición GET al endpoint `/api/patients/{id}/medical-history`, incluyendo el encabezado X-API-KEY para autorizar la consulta.



La API responde con 200 OK y muestra la información clínica del paciente, como diagnóstico, tratamiento y notas médicas.



```
Body Cookies 2 Headers 7 Test Results 200 OK • 309 ms • 558 B
{} JSON Preview Visualize
1 {
2   "message": "Historial médico obtenido correctamente.",
3   "data": {
4     "patient_id": 1,
5     "full_name": "María Fernanda López",
6     "diagnosis": "Hipertensión arterial",
7     "treatment": "Control mensual, dieta baja en sodio y medicamento antihipertensivo.",
8     "medical_notes": "Paciente con antecedentes familiares de enfermedad cardiovascular."
9   }
10 }
```

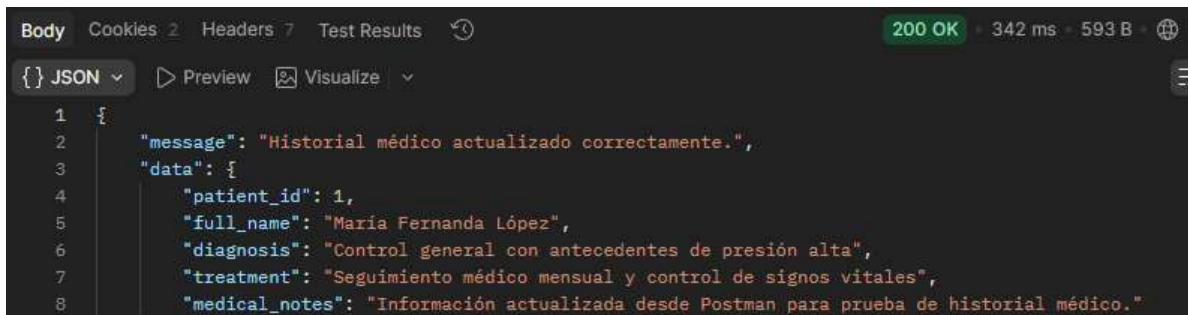
Request 2: Actualizar historial médico

Se realiza una petición POST al endpoint `/api/patients/{id}/medical-history`, enviando en formato JSON el diagnóstico, tratamiento y notas médicas actualizadas.



```
Actividad 11 - API Gestión de Pacientes POST Actualizar historial médico Save Share
POST {{base_url}}/api/patients/1/medical-history Send
Docs Params Authorization Headers 12 Body Scripts Tests Settings Cookies
none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify
1 {
2   "diagnosis": "Control general con antecedentes de presión alta",
3   "treatment": "Seguimiento médico mensual y control de signos vitales",
4   "medical_notes": "Información actualizada desde Postman para prueba de historial médico."
5 }
```

El sistema responde con 200 OK, confirmando que la información del historial médico fue actualizada correctamente.



```
Body Cookies 2 Headers 7 Test Results 200 OK • 342 ms • 593 B
{} JSON Preview Visualize
1 {
2   "message": "Historial médico actualizado correctamente.",
3   "data": {
4     "patient_id": 1,
5     "full_name": "María Fernanda López",
6     "diagnosis": "Control general con antecedentes de presión alta",
7     "treatment": "Seguimiento médico mensual y control de signos vitales",
8     "medical_notes": "Información actualizada desde Postman para prueba de historial médico."
9   }
10 }
```

Request 3: Validar historial médico con datos incompletos

Se envía una petición POST con campos obligatorios vacíos, como diagnosis y treatment, para validar el control de errores.



```
Actividad 11 - API Gestión de Pacientes POST Historial médico con datos inválidos Save Share
POST {{base_url}}/api/patients/1/medical-history Send
Docs Params Authorization Headers 12 Body Scripts Tests Settings Cookies
none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify
1 {
2   "diagnosis": "",
3   "treatment": "",
4   "medical_notes": "Intento de guardar historial médico incompleto."
5 }
```

La API responde con 422 Unprocessable Entity, indicando que los campos obligatorios del historial médico no pueden quedar vacíos.

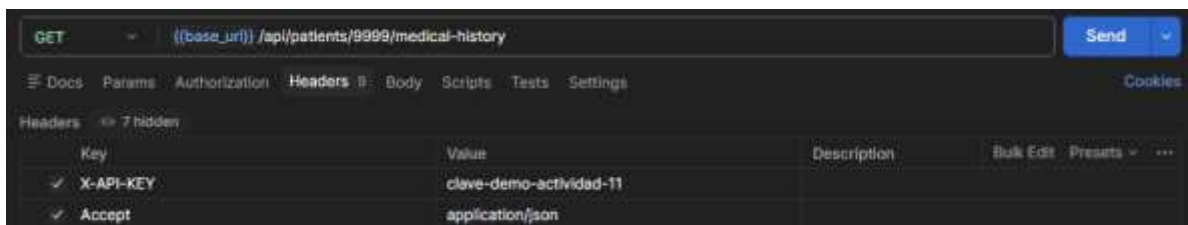


The screenshot shows a REST client interface with the 'Body' tab selected. The response is a JSON object indicating a 422 Unprocessable Content error. The JSON structure is as follows:

```
1 {
2   "message": "The diagnosis field is required. (and 1 more error)",
3   "errors": {
4     "diagnosis": [
5       "The diagnosis field is required."
6     ],
7     "treatment": [
8       "The treatment field is required."
9     ]
10  }
11 }
```

Request 4: Consultar historial médico de paciente inexistente

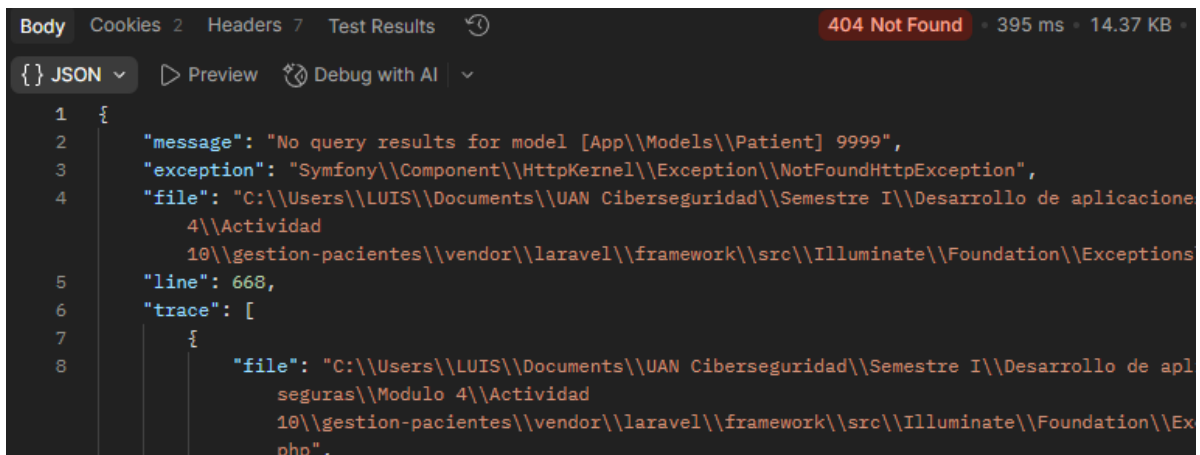
Se realiza una petición GET al historial médico de un paciente con un ID inexistente, por ejemplo /api/patients/9999/medical-history.



The screenshot shows a REST client interface with the 'Headers' tab selected. The request is a GET to the endpoint /api/patients/9999/medical-history. The headers are as follows:

Key	Value	Description
X-API-KEY	clave-demo-actividad-11	
Accept	application/json	

Aparece 404 Not Found. Esto demuestra que la API controla el caso en el que se intenta consultar un paciente que no existe.



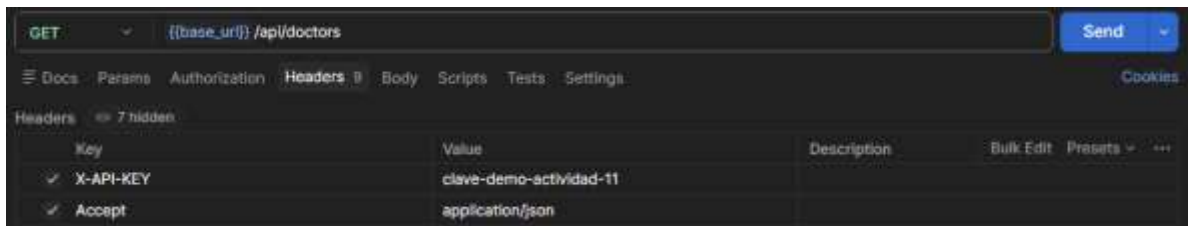
The screenshot shows a REST client interface with the 'Body' tab selected. The response is a JSON object indicating a 404 Not Found error. The JSON structure is as follows:

```
1 {
2   "message": "No query results for model [App\\Models\\Patient] 9999",
3   "exception": "Symfony\\Component\\HttpKernel\\Exception\\NotFoundHttpException",
4   "file": "C:\\Users\\LUIS\\Documents\\UAN Ciberseguridad\\Semestre I\\Desarrollo de aplicacione
5     4\\Actividad
6     10\\gestion-pacientes\\vendor\\laravel\\framework\\src\\Illuminate\\Foundation\\Exceptions
7   "line": 668,
8   "trace": [
9     {
10      "file": "C:\\Users\\LUIS\\Documents\\UAN Ciberseguridad\\Semestre I\\Desarrollo de apl
11        seguras\\Modulo 4\\Actividad
12        10\\gestion-pacientes\\vendor\\laravel\\framework\\src\\Illuminate\\Foundation\\Exp
13        php",
```

Endpoints de Doctores en Postman

Request 1: Listar doctores

Se configura una petición GET al endpoint `/api/doctors`, incluyendo el encabezado X-API-KEY para autorizar la consulta.

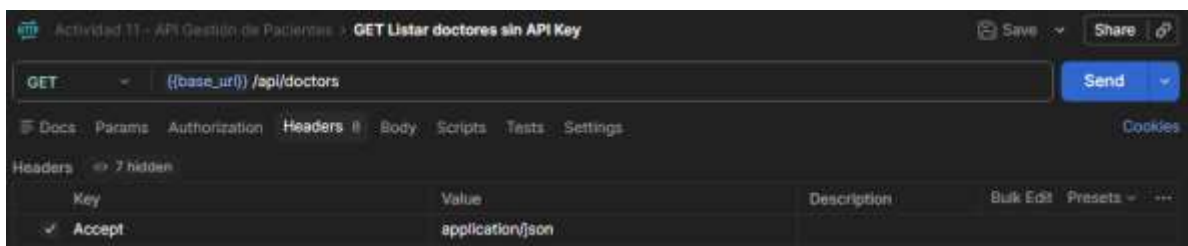


La API responde con 200 OK y muestra el listado de usuarios registrados con rol doctor.

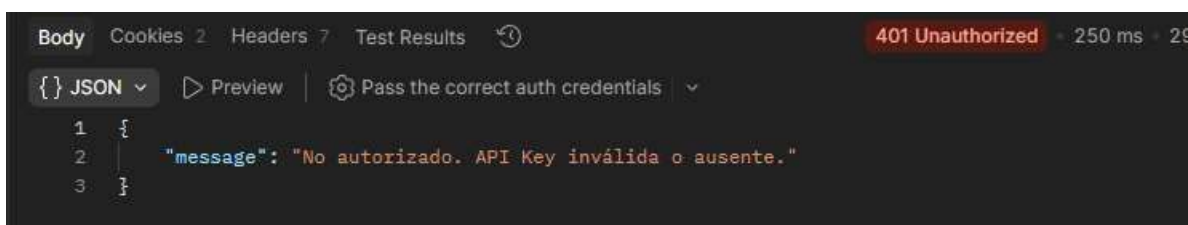


Request 2: Probar endpoint de doctores sin API Key

Se envía una petición GET al endpoint `/api/doctors` sin incluir el encabezado X-API-KEY.



El sistema responde con 401 Unauthorized, confirmando que el endpoint está protegido y no permite acceso sin clave de autorización.

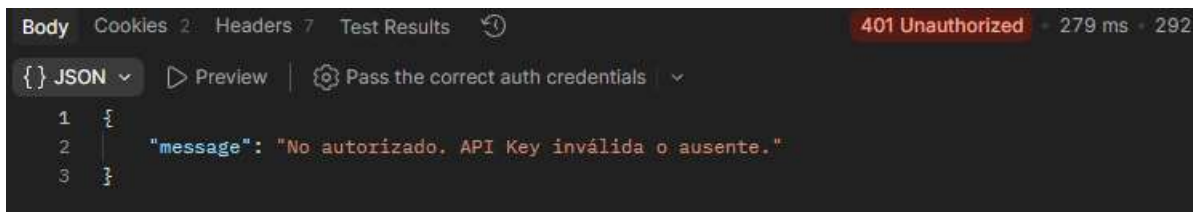


Request 3: Probar endpoint de doctores con API Key incorrecta

Se realiza una petición GET al endpoint /api/doctors, pero enviando una API Key incorrecta.



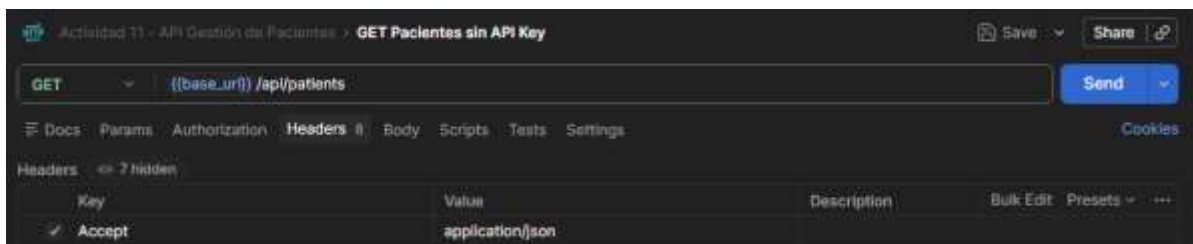
La API responde con 401 Unauthorized, indicando que la clave enviada no es válida y que la solicitud fue rechazada.



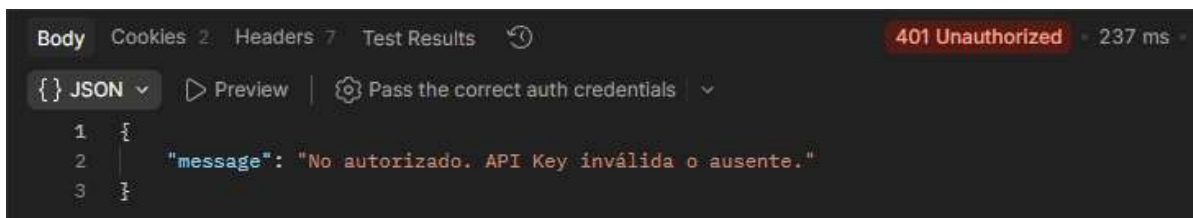
Pruebas de seguridad y manejo de errores en Postman

Request 1: Acceso sin API Key

Se envía una petición GET al endpoint /api/patients sin incluir el encabezado X-API-KEY.



La API responde con 401 Unauthorized, indicando que no se permite acceder al endpoint sin una clave de autorización válida.

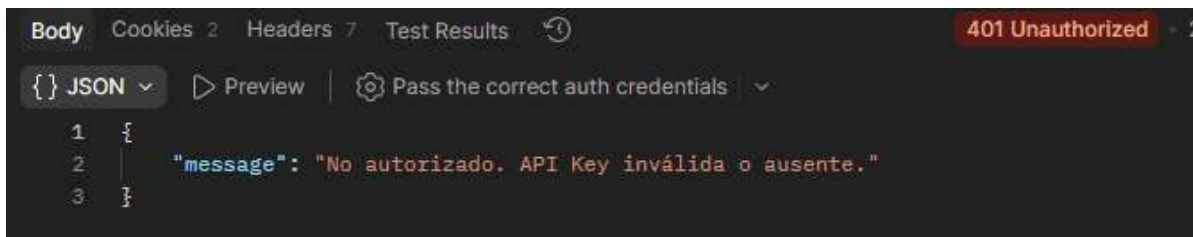


Request 2: Acceso con API Key incorrecta

Se realiza una petición GET al endpoint `/api/patients`, pero enviando una API Key incorrecta en el encabezado.

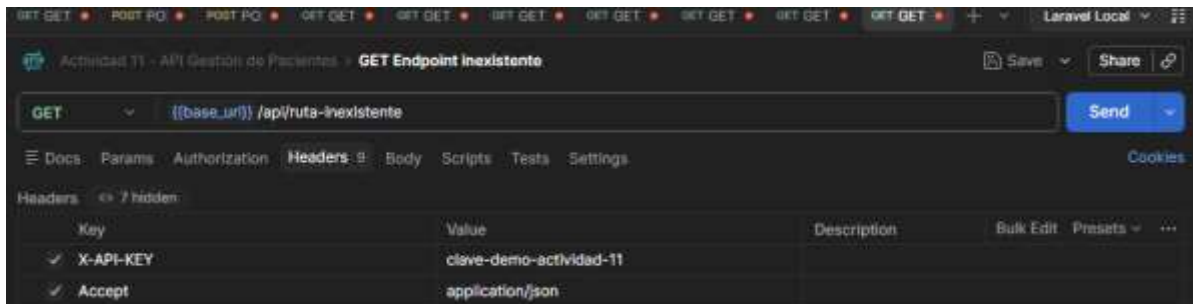


El sistema responde con 401 Unauthorized, confirmando que la clave enviada no es válida y que la solicitud fue rechazada.

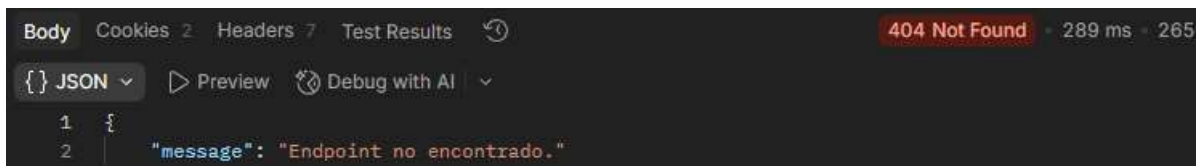


Request 3: Endpoint inexistente

Se realiza una petición GET a una ruta API que no existe, como `/api/ruta-inexistente`.

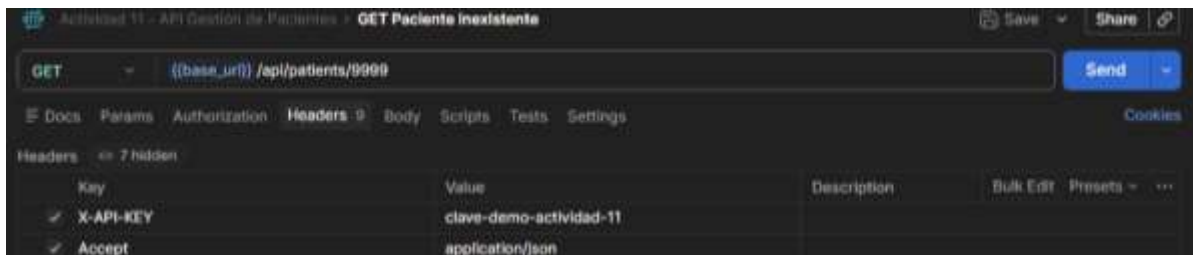


La API responde con 404 Not Found y muestra un mensaje indicando que el endpoint no fue encontrado.

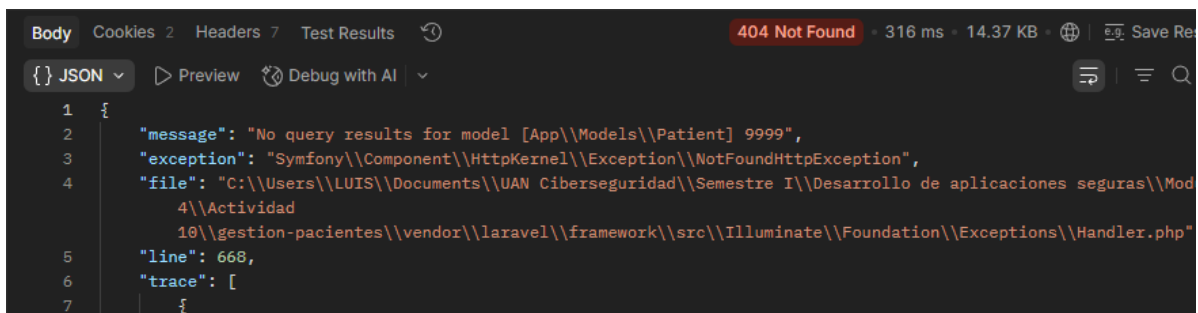


Request 4: Paciente inexistente

Se consulta un paciente con un ID inexistente, por ejemplo /api/patients/9999.

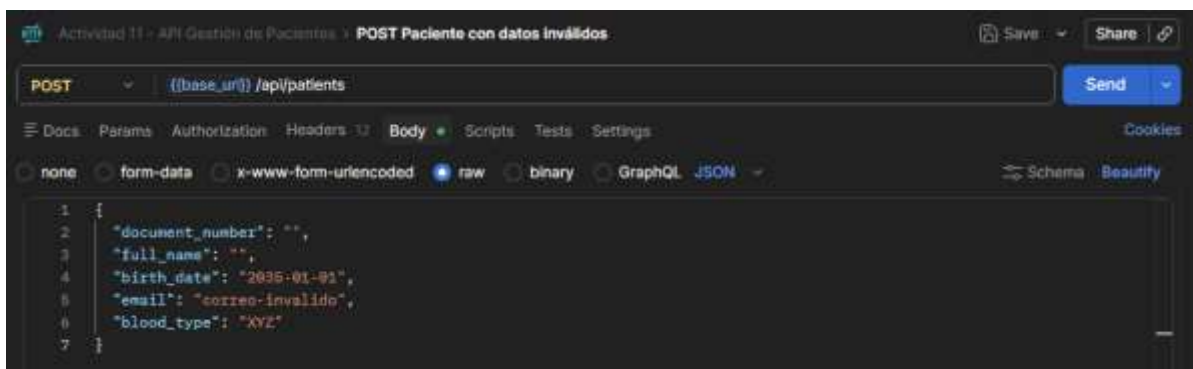


El sistema responde con 404 Not Found, demostrando que la API controla correctamente la consulta de recursos que no existen.

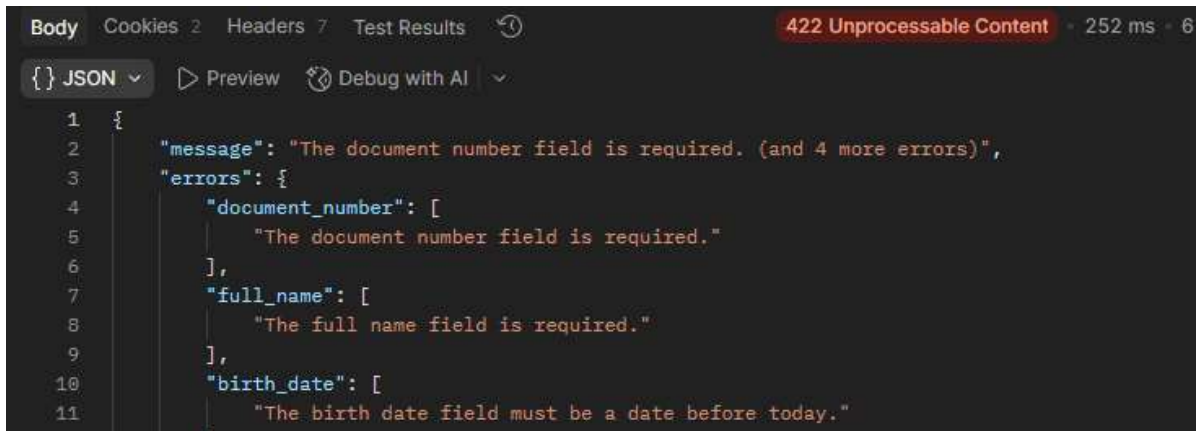


Request 5: Crear paciente con datos inválidos

Se envía una petición POST al endpoint /api/patients con campos vacíos o datos inválidos, como correo incorrecto o fecha no permitida.



La API responde con 422 Unprocessable Entity, mostrando errores de validación en los campos enviados.



The screenshot shows a REST client interface with the 'Body' tab selected. The response is a JSON object indicating a 422 Unprocessable Content error. The message states that the document number field is required, along with four other errors. The errors are listed in an 'errors' object, with 'document_number' being the first one. The other errors are for 'full_name' and 'birth_date'.

```
1 {
2   "message": "The document number field is required. (and 4 more errors)",
3   "errors": {
4     "document_number": [
5       "The document number field is required."
6     ],
7     "full_name": [
8       "The full name field is required."
9     ],
10    "birth_date": [
11      "The birth date field must be a date before today."
12    ]
13  }
14 }
```

Request 6: Historial médico incompleto

Se envía una petición POST al endpoint `/api/patients/{id}/medical-history` dejando vacíos campos obligatorios como diagnóstico y tratamiento.



The screenshot shows a REST client interface with the 'Body' tab selected. The request is a POST to the endpoint `/api/patients/1/medical-history`. The body is a JSON object with empty 'diagnosis' and 'treatment' fields, and a 'medical_notes' field containing a Spanish comment about testing with incomplete fields.

```
1 {
2   "diagnosis": "",
3   "treatment": "",
4   "medical_notes": "Prueba de validación con campos incompletos."
5 }
```

El sistema responde con 422 Unprocessable Entity, indicando que los campos requeridos del historial médico deben completarse.

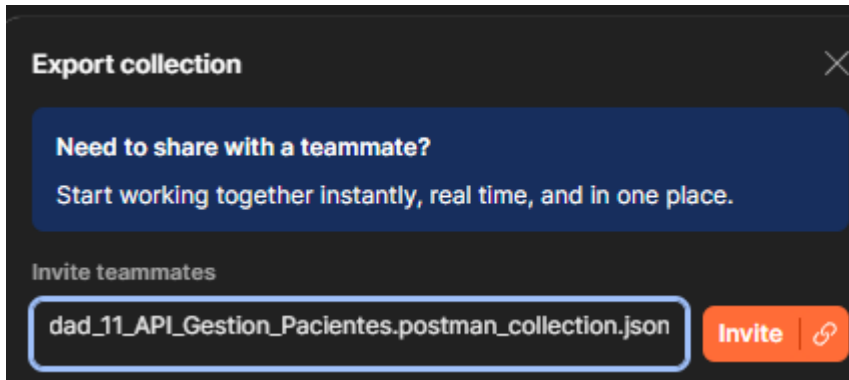


The screenshot shows a REST client interface with the 'Body' tab selected. The response is a JSON object indicating a 422 Unprocessable Content error. The message states that the diagnosis field is required, along with one more error. The errors are listed in an 'errors' object, with 'diagnosis' being the first one. The other error is for 'treatment'.

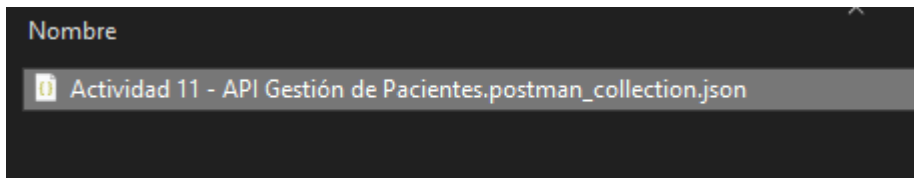
```
1 {
2   "message": "The diagnosis field is required. (and 1 more error)",
3   "errors": {
4     "diagnosis": [
5       "The diagnosis field is required."
6     ],
7     "treatment": [
8       "The treatment field is required."
9     ]
10  }
11 }
```

Exportar la Colección de Postman para GitHub

Para que también quede evidencia en el repositorio, se exporta la colección de Postman.

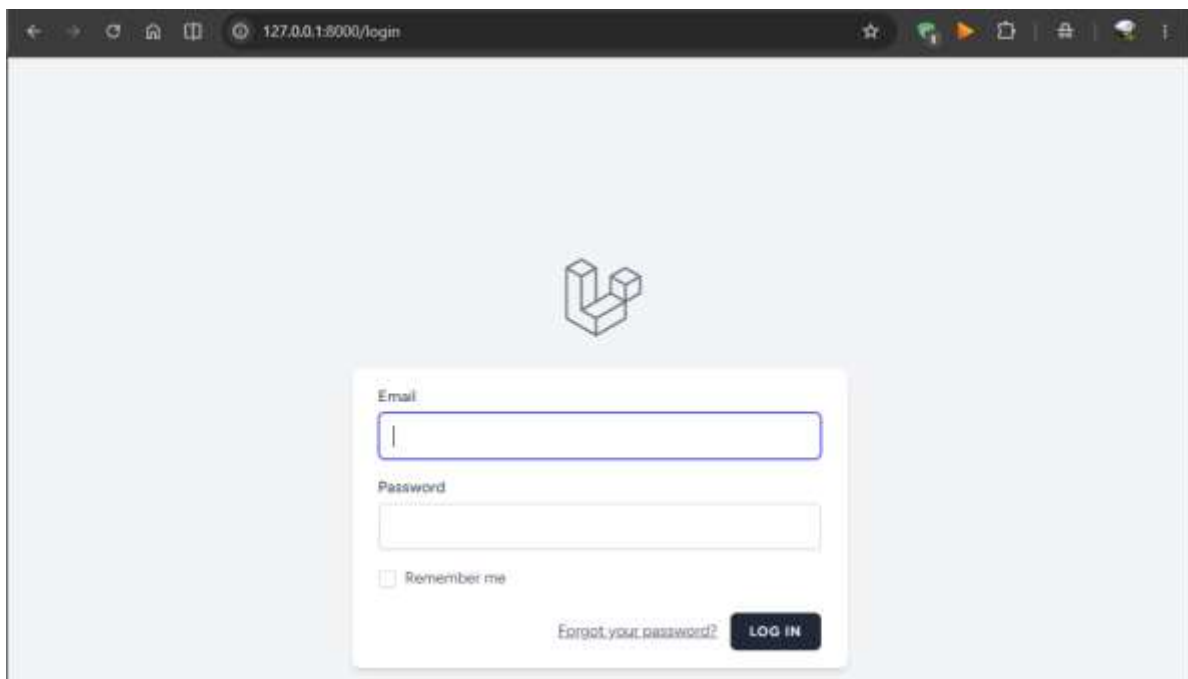


Se crea una carpeta en el proyecto de gestión de pacientes y se guarda en esa ubicación.

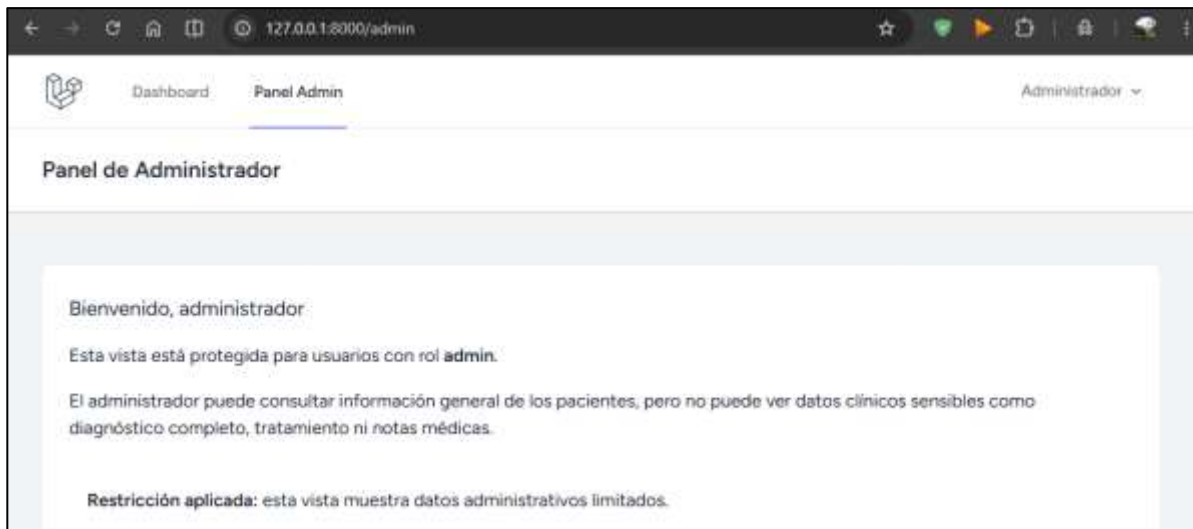


Exploración y Pruebas de Flujos Realistas Combinando Frontend y Backend

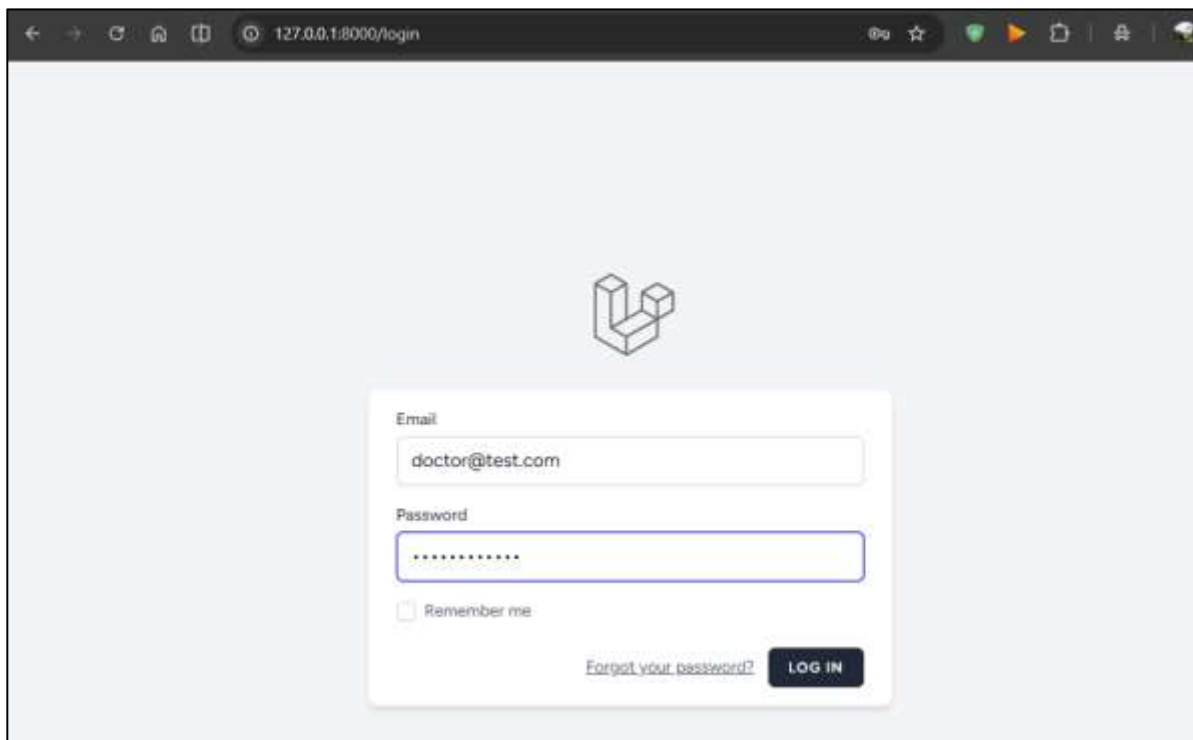
Flujo 1: Login y acceso como administrador



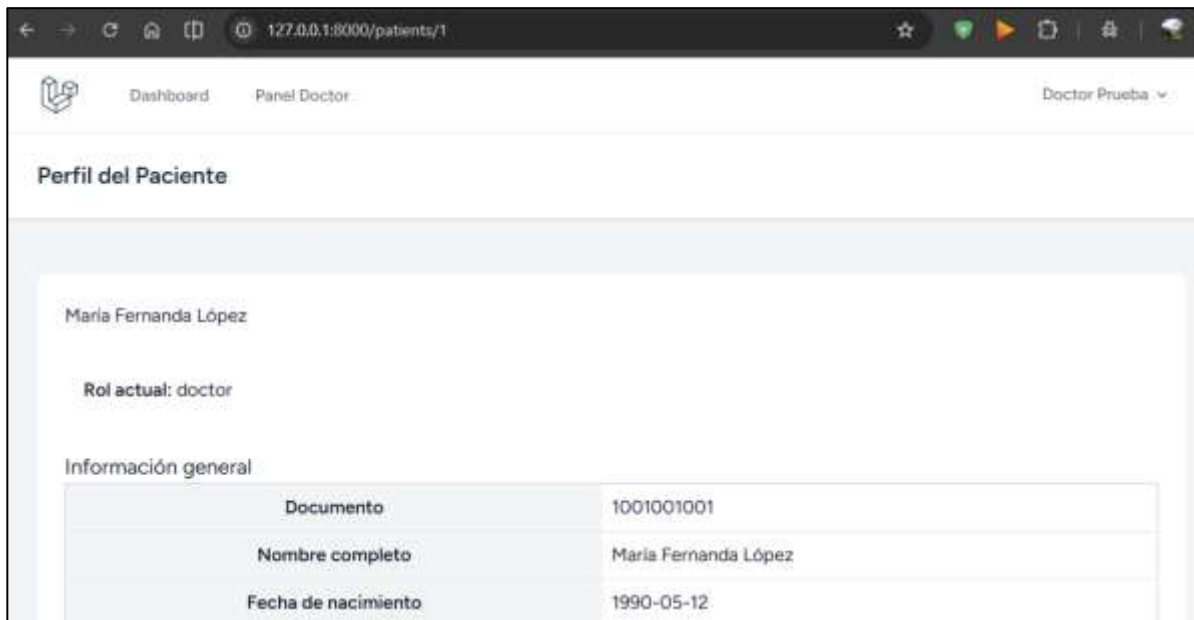
El administrador accede correctamente, pero no ve información clínica sensible.



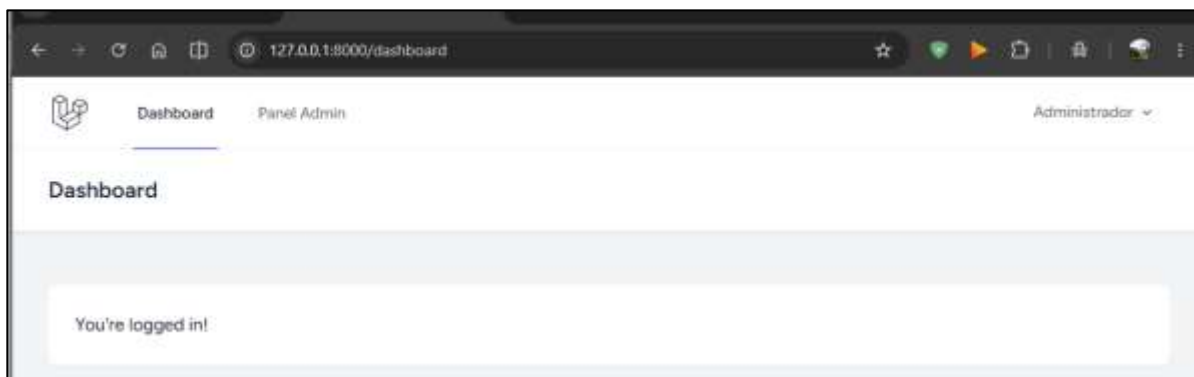
Flujo 2: Login y acceso como doctor.



El doctor puede ver información clínica del paciente, como diagnóstico, tratamiento y notas médicas.



Flujo 3: Acceso no autorizado

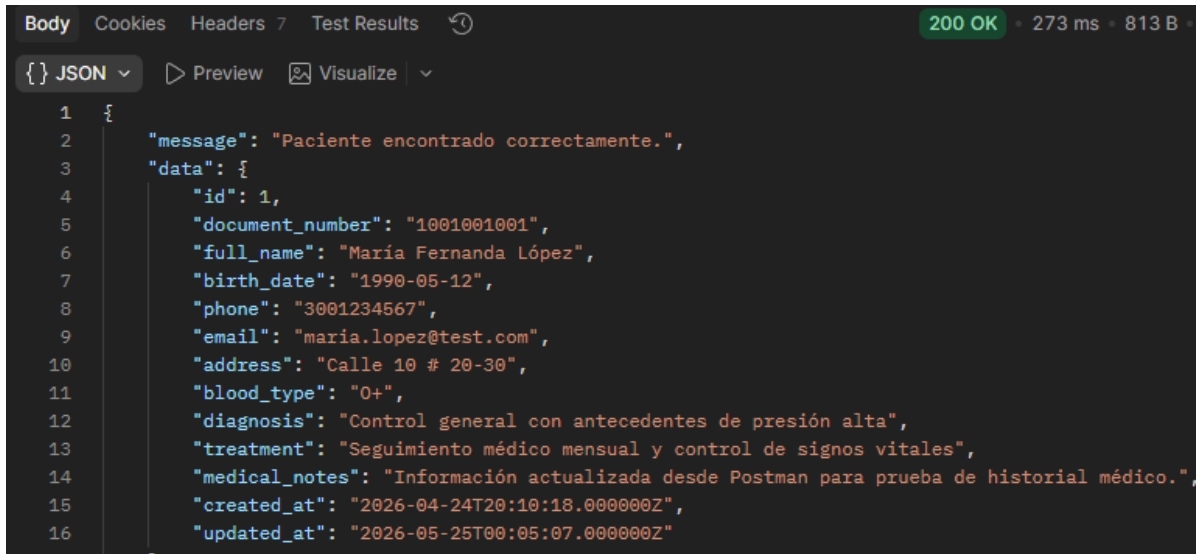


El sistema muestra error 403 o mensaje de acceso no autorizado.



Flujo 4: Consulta por API y validación en frontend

En Postman se ejecuta: GET {{base_url}}/api/patients/1



```
1 {
2   "message": "Paciente encontrado correctamente.",
3   "data": {
4     "id": 1,
5     "document_number": "1001001001",
6     "full_name": "María Fernanda López",
7     "birth_date": "1990-05-12",
8     "phone": "3001234567",
9     "email": "maria.lopez@test.com",
10    "address": "Calle 10 # 20-30",
11    "blood_type": "O+",
12    "diagnosis": "Control general con antecedentes de presión alta",
13    "treatment": "Seguimiento médico mensual y control de signos vitales",
14    "medical_notes": "Información actualizada desde Postman para prueba de historial médico.",
15    "created_at": "2026-04-24T20:10:18.000000Z",
16    "updated_at": "2026-05-25T00:05:07.000000Z"
17  }
```

La información consultada por API corresponde con la mostrada en la interfaz.



Documento	
Nombre completo	1001001001
Fecha de nacimiento	María Fernanda López
	1990-05-12

Vista completa para doctor

El doctor puede consultar información clínica protegida del paciente.

Información de contacto

Teléfono	3001234567
Correo	maria.lopez@test.com
Dirección	Calle 10 # 20-30

Información clínica protegida

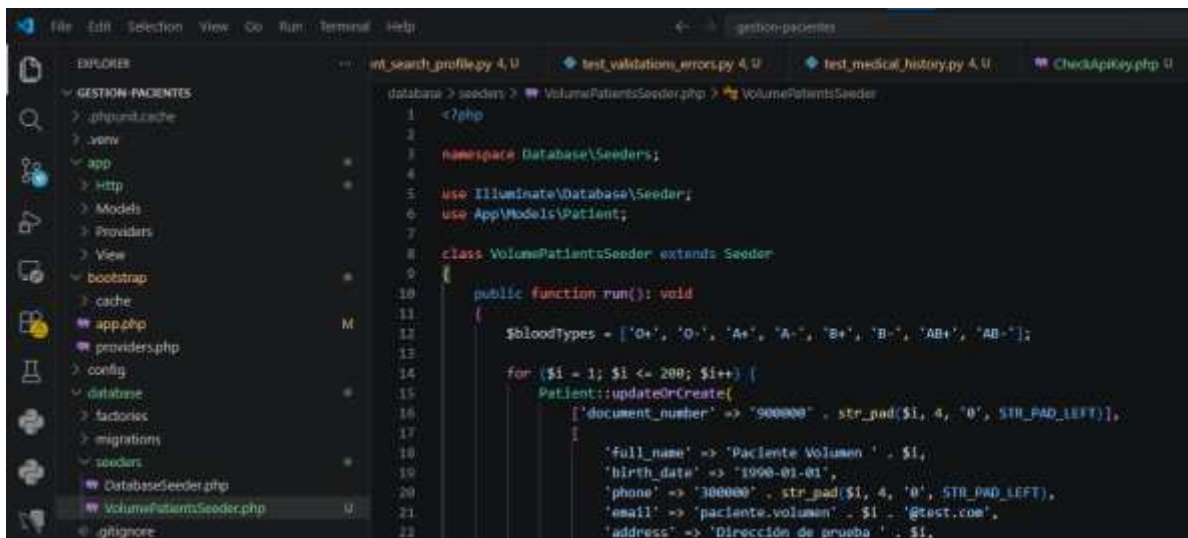
Tipo de sangre	O+
Diagnóstico	Control general con antecedentes de presión alta
Tratamiento	Seguimiento médico mensual y control de signos vitales
Notas médicas	Información actualizada desde Postman para prueba de historial médico.

Pruebas De Volumen Con Base De Datos Llena

Se crea un Seeder en VS Code.

```
php artisan make:seeder VolumePatientsSeeder  
on-pacientes\database\seeders\VolumePatientsSeeder.php] created succe
```

Esto crea el archivo VolumePatientsSeeder.php el cual genera registros con datos básicos y clínicos simulados, y se reemplaza con lo siguiente.



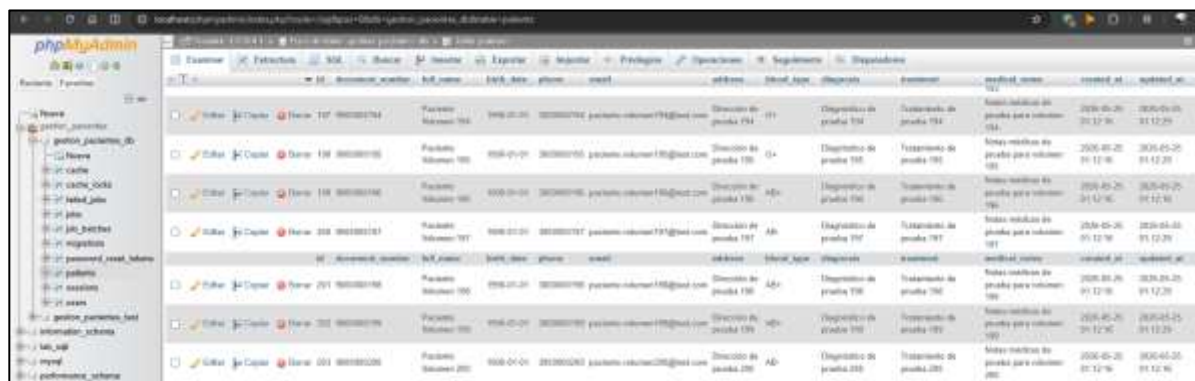
```
1 <?php  
2  
3 namespace Database\Seeders;  
4  
5 use Illuminate\Database\Seeder;  
6 use App\Models\Patient;  
7  
8 class VolumePatientsSeeder extends Seeder  
9 {  
10     public function run(): void  
11     {  
12         $bloodTypes = ['O+', 'O-', 'A+', 'A-', 'B+', 'B-', 'AB+', 'AB-'];  
13  
14         for ($i = 1; $i <= 200; $i++) {  
15             Patient::updateOrCreate(  
16                 ['document_number' => "9000000" . str_pad($i, 4, "0", STR_PAD_LEFT)],  
17                 [  
18                     'full_name' => 'Paciente Volumen ' . $i,  
19                     'birth_date' => "1990-01-01",  
20                     'phone' => "3000000" . str_pad($i, 4, "0", STR_PAD_LEFT),  
21                     'email' => "paciente.volumen" . $i . "@test.com",  
22                     'address' => "Dirección de prueba" . $i,  
23                 ]  
24             );  
25         }  
26     }  
27 }
```

En la terminal se ejecuta el seeder

```
php artisan db:seed --class=VolumePatientsSeeder
```

INFO Seeding database.

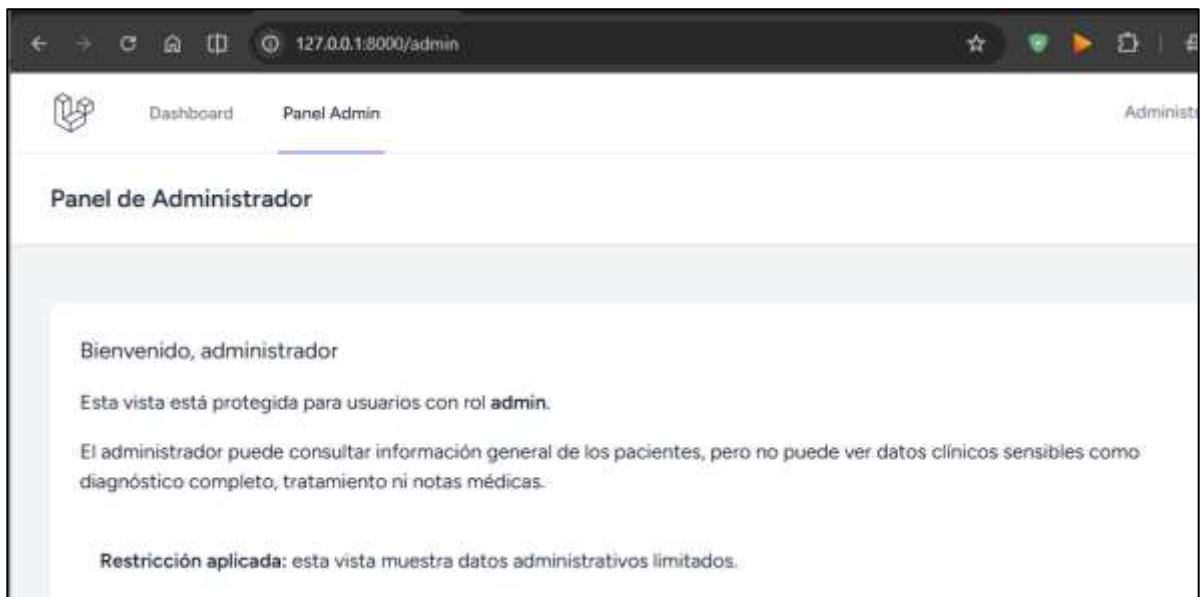
En phpMyAdmin la tabla patients aparecen 200 registros nuevos.



document_number	full_name	birth_date	phone	email	address	blood_type	diagnosis	treatment	medical_notes	created_at	updated_at
9000000000000001	Paciente Volumen 1	1990-01-01	3000000000000001	paciente.volumen1@test.com	Dirección de prueba 1	O+	Diagnóstico de prueba 1	Tratamiento de prueba 1	Notas médicas de prueba para volumen 1	2024-05-20 00:12:30	2024-05-20 00:12:30
9000000000000002	Paciente Volumen 2	1990-01-01	3000000000000002	paciente.volumen2@test.com	Dirección de prueba 2	O+	Diagnóstico de prueba 2	Tratamiento de prueba 2	Notas médicas de prueba para volumen 2	2024-05-20 00:12:30	2024-05-20 00:12:30
9000000000000003	Paciente Volumen 3	1990-01-01	3000000000000003	paciente.volumen3@test.com	Dirección de prueba 3	O+	Diagnóstico de prueba 3	Tratamiento de prueba 3	Notas médicas de prueba para volumen 3	2024-05-20 00:12:30	2024-05-20 00:12:30
9000000000000004	Paciente Volumen 4	1990-01-01	3000000000000004	paciente.volumen4@test.com	Dirección de prueba 4	O+	Diagnóstico de prueba 4	Tratamiento de prueba 4	Notas médicas de prueba para volumen 4	2024-05-20 00:12:30	2024-05-20 00:12:30
9000000000000005	Paciente Volumen 5	1990-01-01	3000000000000005	paciente.volumen5@test.com	Dirección de prueba 5	O+	Diagnóstico de prueba 5	Tratamiento de prueba 5	Notas médicas de prueba para volumen 5	2024-05-20 00:12:30	2024-05-20 00:12:30
9000000000000006	Paciente Volumen 6	1990-01-01	3000000000000006	paciente.volumen6@test.com	Dirección de prueba 6	O+	Diagnóstico de prueba 6	Tratamiento de prueba 6	Notas médicas de prueba para volumen 6	2024-05-20 00:12:30	2024-05-20 00:12:30
9000000000000007	Paciente Volumen 7	1990-01-01	3000000000000007	paciente.volumen7@test.com	Dirección de prueba 7	O+	Diagnóstico de prueba 7	Tratamiento de prueba 7	Notas médicas de prueba para volumen 7	2024-05-20 00:12:30	2024-05-20 00:12:30
9000000000000008	Paciente Volumen 8	1990-01-01	3000000000000008	paciente.volumen8@test.com	Dirección de prueba 8	O+	Diagnóstico de prueba 8	Tratamiento de prueba 8	Notas médicas de prueba para volumen 8	2024-05-20 00:12:30	2024-05-20 00:12:30
9000000000000009	Paciente Volumen 9	1990-01-01	3000000000000009	paciente.volumen9@test.com	Dirección de prueba 9	O+	Diagnóstico de prueba 9	Tratamiento de prueba 9	Notas médicas de prueba para volumen 9	2024-05-20 00:12:30	2024-05-20 00:12:30
9000000000000010	Paciente Volumen 10	1990-01-01	3000000000000010	paciente.volumen10@test.com	Dirección de prueba 10	O+	Diagnóstico de prueba 10	Tratamiento de prueba 10	Notas médicas de prueba para volumen 10	2024-05-20 00:12:30	2024-05-20 00:12:30
9000000000000011	Paciente Volumen 11	1990-01-01	3000000000000011	paciente.volumen11@test.com	Dirección de prueba 11	O+	Diagnóstico de prueba 11	Tratamiento de prueba 11	Notas médicas de prueba para volumen 11	2024-05-20 00:12:30	2024-05-20 00:12:30
9000000000000012	Paciente Volumen 12	1990-01-01	3000000000000012	paciente.volumen12@test.com	Dirección de prueba 12	O+	Diagnóstico de prueba 12	Tratamiento de prueba 12	Notas médicas de prueba para volumen 12	2024-05-20 00:12:30	2024-05-20 00:12:30
9000000000000013	Paciente Volumen 13	1990-01-01	3000000000000013	paciente.volumen13@test.com	Dirección de prueba 13	O+	Diagnóstico de prueba 13	Tratamiento de prueba 13	Notas médicas de prueba para volumen 13	2024-05-20 00:12:30	2024-05-20 00:12:30
9000000000000014	Paciente Volumen 14	1990-01-01	3000000000000014	paciente.volumen14@test.com	Dirección de prueba 14	O+	Diagnóstico de prueba 14	Tratamiento de prueba 14	Notas médicas de prueba para volumen 14	2024-05-20 00:12:30	2024-05-20 00:12:30
9000000000000015	Paciente Volumen 15	1990-01-01	3000000000000015	paciente.volumen15@test.com	Dirección de prueba 15	O+	Diagnóstico de prueba 15	Tratamiento de prueba 15	Notas médicas de prueba para volumen 15	2024-05-20 00:12:30	2024-05-20 00:12:30
9000000000000016	Paciente Volumen 16	1990-01-01	3000000000000016	paciente.volumen16@test.com	Dirección de prueba 16	O+	Diagnóstico de prueba 16	Tratamiento de prueba 16	Notas médicas de prueba para volumen 16	2024-05-20 00:12:30	2024-05-20 00:12:30
9000000000000017	Paciente Volumen 17	1990-01-01	3000000000000017	paciente.volumen17@test.com	Dirección de prueba 17	O+	Diagnóstico de prueba 17	Tratamiento de prueba 17	Notas médicas de prueba para volumen 17	2024-05-20 00:12:30	2024-05-20 00:12:30
9000000000000018	Paciente Volumen 18	1990-01-01	3000000000000018	paciente.volumen18@test.com	Dirección de prueba 18	O+	Diagnóstico de prueba 18	Tratamiento de prueba 18	Notas médicas de prueba para volumen 18	2024-05-20 00:12:30	2024-05-20 00:12:30
9000000000000019	Paciente Volumen 19	1990-01-01	3000000000000019	paciente.volumen19@test.com	Dirección de prueba 19	O+	Diagnóstico de prueba 19	Tratamiento de prueba 19	Notas médicas de prueba para volumen 19	2024-05-20 00:12:30	2024-05-20 00:12:30
9000000000000020	Paciente Volumen 20	1990-01-01	3000000000000020	paciente.volumen20@test.com	Dirección de prueba 20	O+	Diagnóstico de prueba 20	Tratamiento de prueba 20	Notas médicas de prueba para volumen 20	2024-05-20 00:12:30	2024-05-20 00:12:30

Se ejecuta la aplicación con la base de datos cargada y se prueba manualmente:

<http://127.0.0.1:8000/admin>



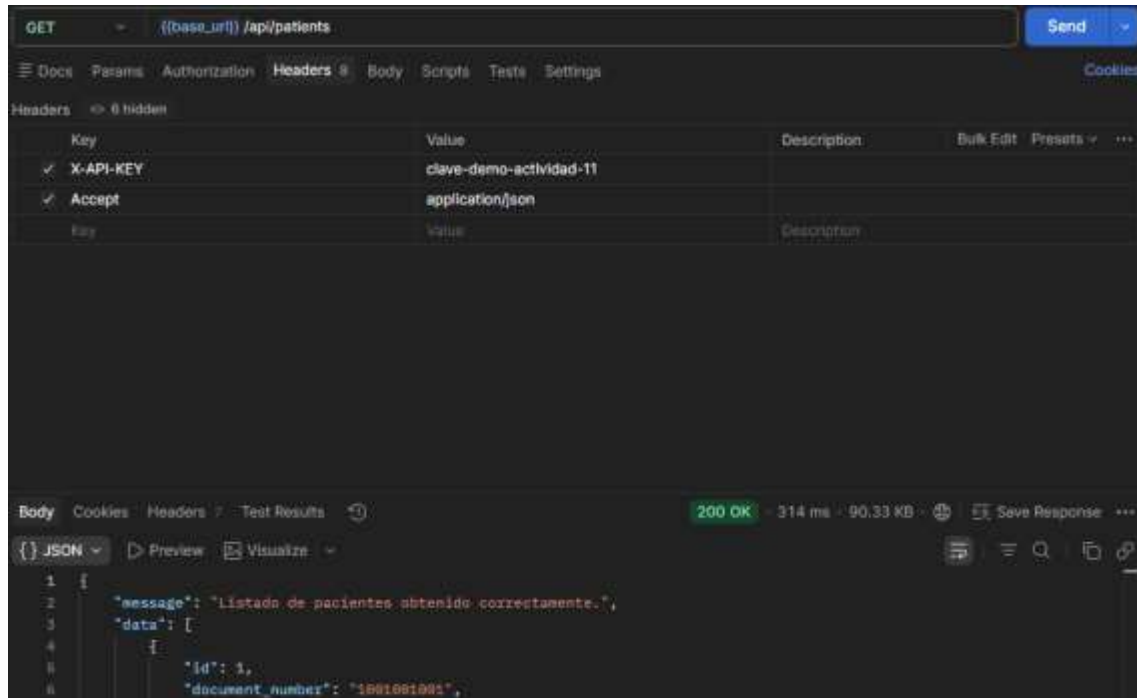
<http://127.0.0.1:8000/doctor>



<http://127.0.0.1:8000/patients/1>



En postman se prueba GET {{base_url}}/api/patients



La prueba permitió comprobar que la aplicación puede manejar una base de datos con múltiples registros de pacientes sin presentar errores visibles en los flujos principales. Esto evidencia que el sistema mantiene su funcionamiento básico al trabajar con mayor volumen de datos.

Pruebas De Seguridad y Permisos

La primera prueba es con un usuario no autenticado que intenta entrar a rutas protegidas

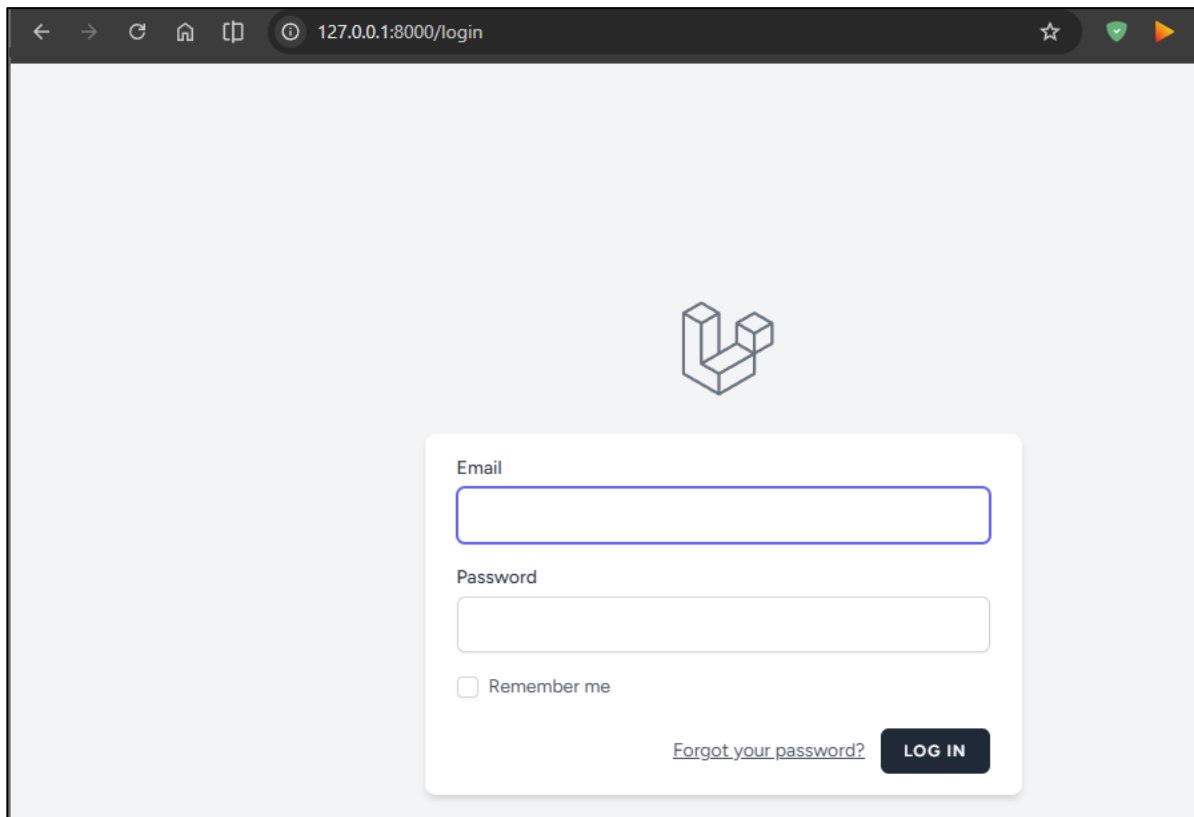
`http://127.0.0.1:8000/admin`

`http://127.0.0.1:8000/doctor`

`http://127.0.0.1:8000/patients/1`



El resultado es que el sistema lo redirige al login en los tres casos.



127.0.0.1:8000/login



Email

Password

☐ Remember me

[Forgot your password?](#) **LOG IN**

Otra prueba es como usuario Admin que intenta acceder al panel doctor.

Como resultado, el sistema muestra error **403** o acceso no autorizado



127.0.0.1:8000/doctor

403

Acceso no autorizado

No tienes permisos suficientes para acceder a esta sección. Esta restricción protege la información clínica de los pacientes.

Motivo: el sistema detectó que tu rol no tiene autorización para consultar este recurso.

La siguiente prueba es como usuario Doctor, intentar acceder al panel admin.

El resultado es que el sistema muestra error 403 o acceso no autorizado.



Otra prueba consiste que el Admin no puede ver la información clínica completa.

Como resultado, el admin solo ve datos básicos, pero no diagnóstico, tratamiento ni notas médicas completas.



La siguiente prueba es ver si el Doctor puede ver información clínica.

Como resultado el doctor ve información clínica del paciente.



Nombre completo	Maria Fernanda López
Fecha de nacimiento	1990-05-12

Vista completa para doctor
El doctor puede consultar información clínica protegida del paciente.

Información de contacto

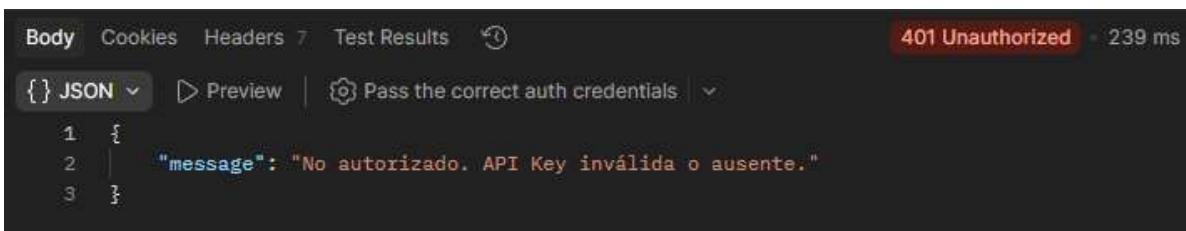
Teléfono	3001234567
Correo	maria.lopez@test.com
Dirección	Calle 10 # 20-30

Información clínica protegida

Tipo de sangre	O+
Diagnóstico	Control general con antecedentes de presión alta
Tratamiento	Seguimiento médico mensual y control de signos vitales
Notas médicas	Información actualizada desde Postman para prueba de historial médico.

Y la última prueba es la API sin clave de seguridad en Postman.

El resultado es 401 Unauthorized.



```
Body Cookies Headers 7 Test Results 401 Unauthorized 239 ms
{ } JSON Preview Pass the correct auth credentials
1 {
2   "message": "No autorizado. API Key inválida o ausente."
3 }
```

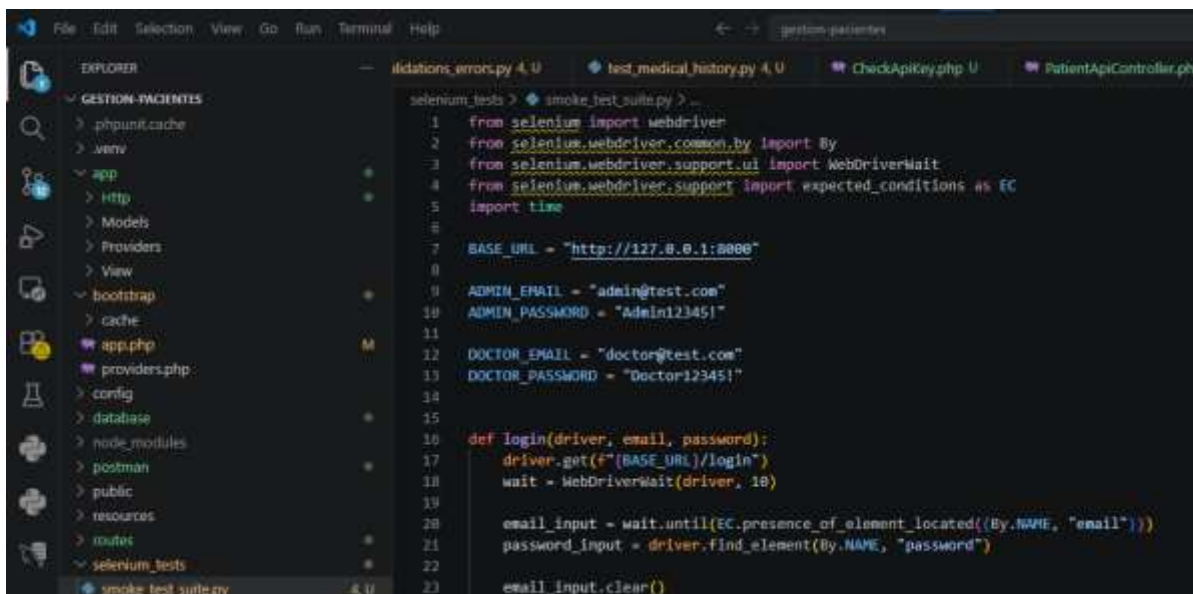
Los resultados evidencian que la aplicación aplica controles adecuados de autenticación y autorización. Además, se comprobó que los datos clínicos sensibles no son visibles para usuarios sin permisos y que los endpoints API protegidos rechazan solicitudes no autorizadas.

Smoke Test Suite Crítico

Se crea un solo script de Selenium que valide los flujos mínimos más importantes:

- La página de login carga.
- El admin puede iniciar sesión y entrar a /admin.
- El doctor puede iniciar sesión y entrar a /doctor.
- Un usuario sin sesión no puede entrar a /patients/1.
- El doctor puede ver el perfil clínico del paciente.

En la carpeta selenium_tests, se crea el archivo: smoke_test_suite.py



```
1 from selenium import webdriver
2 from selenium.webdriver.common.by import By
3 from selenium.webdriver.support.ui import WebDriverWait
4 from selenium.webdriver.support import expected_conditions as EC
5 import time
6
7 BASE_URL = "http://127.0.0.1:8080"
8
9 ADMIN_EMAIL = "admin@test.com"
10 ADMIN_PASSWORD = "Admin12345!"
11
12 DOCTOR_EMAIL = "doctor@test.com"
13 DOCTOR_PASSWORD = "Doctor12345!"
14
15
16 def login(driver, email, password):
17     driver.get(f"{BASE_URL}/login")
18     wait = WebDriverWait(driver, 10)
19
20     email_input = wait.until(EC.presence_of_element_located((By.NAME, "email")))
21     password_input = driver.find_element(By.NAME, "password")
22
23     email_input.clear()
```

Se ejecuta desde la tyterminal

```
python selenium_tests/smoke_test_suite.py
```

El resultado es el correcto en los Smoke Test definidos para validar rápidamente los flujos principales de la aplicación.

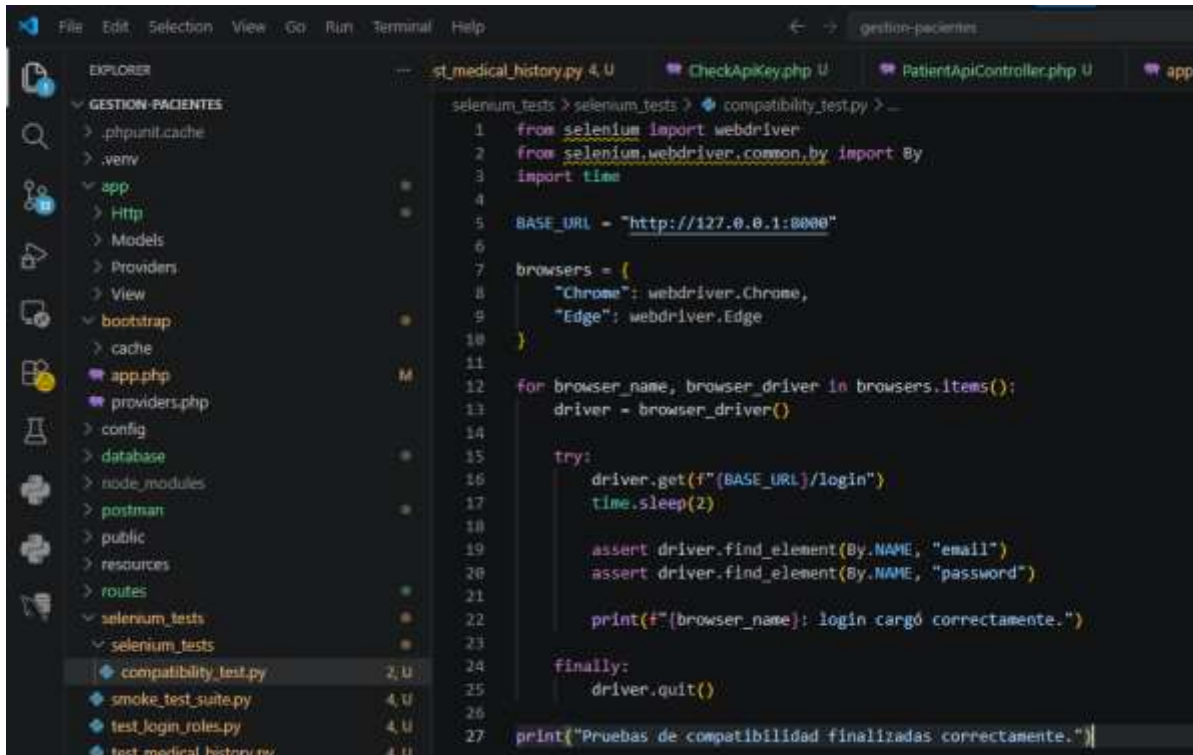
```
SMOKE-001: Página de login cargó correctamente.
SMOKE-002: Admin accedió correctamente al panel.
SMOKE-003: Doctor accedió correctamente al panel.
SMOKE-004: Usuario no autenticado fue redirigido al login.
SMOKE-005: Doctor visualizó correctamente el perfil clínico del paciente.
Smoke test suite crítico finalizado correctamente.
```

El smoke test permitió validar los flujos mínimos necesarios para confirmar que la aplicación está operativa. Los resultados fueron satisfactorios, ya que el sistema permitió el acceso según el rol, protegió rutas privadas y mostró correctamente la información clínica al usuario autorizado.

Pruebas de Compatibilidad con Navegadores

En Selenium se crea un archivo `compatibility_test.py` para probar la funcionalidad del aplicativo en los navegadores Chrome y Microsoft Edge.

El archivo contiene el siguiente código:



```
1 from selenium import webdriver
2 from selenium.webdriver.common.by import By
3 import time
4
5 BASE_URL = "http://127.0.0.1:8000"
6
7 browsers = {
8     "Chrome": webdriver.Chrome,
9     "Edge": webdriver.Edge
10 }
11
12 for browser_name, browser_driver in browsers.items():
13     driver = browser_driver()
14
15     try:
16         driver.get(f"{BASE_URL}/login")
17         time.sleep(2)
18
19         assert driver.find_element(By.NAME, "email")
20         assert driver.find_element(By.NAME, "password")
21
22         print(f"{browser_name}: login cargó correctamente.")
23
24     finally:
25         driver.quit()
26
27 print("Pruebas de compatibilidad finalizadas correctamente.")
```

Se ejecuta el archivo desde la terminal.

En la pantalla de la terminal apareció correcto.

```
Chrome: login cargó correctamente.
Edge: login cargó correctamente.
Pruebas de compatibilidad finalizadas correctamente.
```

Los resultados permitieron comprobar que la aplicación carga correctamente en los navegadores evaluados. No se identificaron errores visuales o funcionales en la carga inicial del sistema, por lo que la aplicación presenta compatibilidad básica con Chrome y Edge.

Pruebas de Carga y Estrés con JMeter

Se realizó una prueba de carga utilizando Apache JMeter con 20 usuarios virtuales, un tiempo de arranque de 10 segundos y 2 repeticiones por usuario.

Thread Properties

Number of Threads (users): 20

Ramp-up period (seconds): 10

Loop Count: ☐ Infinite 2

La prueba incluyó solicitudes al login, endpoints API de pacientes, doctores, historial médico y acceso al panel administrativo.

Los resultados fueron satisfactorios, ya que se ejecutaron 280 muestras con un porcentaje de error del 0.00%. El tiempo promedio total fue de 4496 ms, con un tiempo mínimo de 246 ms y un tiempo máximo de 9874 ms.

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Send KB/sec	Avg. Bytes
GET Login	40	3813	335	4811	1113.87	0.00%	46.5/sec	5.81	0.08	7968.0
HTTP Request	40	5526	346	8874	2636.23	0.00%	1.3/sec	48.46	0.08	17947.0
GET Panel Admin	40	1471	178	9761	1885.73	0.00%	36.1/sec	5.38	0.14	3147.0
GET API Patients	40	1632	188	3621	770.46	0.00%	35.6/sec	11.38	0.11	62467.0
GET API Doctors	40	1469	1478	3631	896.05	0.00%	23.2/sec	0.23	0.11	4347.0
GET API Medical Hist.	40	3337	1885	4575	983.46	0.00%	23.5/sec	0.34	0.52	3883.0
TOTAL	280	4496	246	9874	2295.31	0.00%	3.8/sec	82.19	0.34	24850.1

Label	# Samples	Average	Median	50% Line	90% Line	95% Line	Max	Maximum	Error %	Throughput	Received KB/sec	Send KB/sec
GET Login	40	3813	1781	4381	4405	4811	375	4811	0.00%	46.5/sec	5.81	0.08
HTTP Request	40	5526	4312	9146	6438	8342	346	9874	0.00%	1.3/sec	48.46	0.08
GET Panel Admin	40	1471	7544	3032	6331	8782	1718	9761	0.00%	36.1/sec	5.38	0.14
GET API Patients	40	1632	3747	4485	4838	3621	1885	3621	0.00%	35.6/sec	11.38	0.11
GET API Doctors	40	1469	3694	4386	4507	3631	3878	3631	0.00%	23.2/sec	0.23	0.11
GET API Medical Hist.	40	3337	3478	4316	4332	4575	1099	4575	0.00%	23.5/sec	0.34	0.52
TOTAL	280	4496	3388	6376	9119	9473	346	9874	0.00%	3.8/sec	82.19	0.34

Esto evidencia que la aplicación respondió correctamente durante la prueba de carga en el entorno local.

También se ejecutó una prueba de estrés utilizando Apache JMeter con 50 usuarios virtuales, un tiempo de arranque de 10 segundos y 2 repeticiones por usuario. Esta prueba tuvo como objetivo observar el comportamiento de la aplicación al recibir una mayor cantidad de solicitudes simultáneas, incluyendo el login, panel administrativo y endpoints API de pacientes, doctores e historial médico.

Thread Properties

Number of Threads (users): 50

Ramp-up period (seconds): 10

Loop Count: ☐ Infinite 2

Durante la prueba se ejecutaron 700 muestras en total, obteniendo un porcentaje de error del 0.00%. El tiempo promedio general fue de 12086 ms, con un tiempo mínimo de 406 ms y un tiempo máximo de 21831 ms.

Label	# Samples	Average	Min	Max	Std. Dev.	Dev. %	Throughput	Received KB/sec	Sent KB/sec	Req. Rate
GET Login	100	7631	454	11963	3383.41	0.30%	33.83/sec	8.72	0.11	1600.0
HTTP Request	200	13898	919	20700	3763.90	0.30%	1.6/sec	44.32	0.21	11987.1
GET Panel Admin	100	20147	10794	21931	374.34	0.30%	26.0/sec	3.77	0.15	9983.3
GET API Patients	100	10393	9953	11149	494.91	0.30%	46.3/sec	61.08	0.13	30497.3
GET API Doctors	100	9719	7611	11123	384.32	0.30%	47.2/sec	5.29	0.13	4344.2
GET API Medical Hist.	100	3842	4622	11393	1792.26	0.30%	42.9/sec	3.41	0.14	3902.0
TOTAL	700	13898	469	21931	3239.92	0.30%	3.6/sec	52.87	0.75	24951.0

Label	# Samples	Average	Median	90% Line	95% Line	99% Line	Min	Maximum	Dev. %	Throughput	Received KB/sec	Sent KB/sec
GET Login	100	7631	5342	12276	13618	11842	469	11963	0.30%	33.83/sec	8.72	0.11
HTTP Request	200	13898	10610	19893	19824	20469	919	20700	0.30%	1.6/sec	44.32	0.21
GET Panel Admin	100	20147	10920	20080	22542	21737	10794	21931	0.30%	26.0/sec	3.77	0.15
GET API Patients	100	10393	10514	11030	11069	11139	9953	11149	0.30%	46.3/sec	61.08	0.13
GET API Doctors	100	9719	8862	10800	11004	11897	7611	11123	0.30%	47.2/sec	5.29	0.13
GET API Medical Hist.	100	3842	3981	10012	10979	11579	4622	11393	0.30%	42.9/sec	3.41	0.14
TOTAL	700	13898	9989	19930	20138	21444	469	21931	0.30%	3.6/sec	52.87	0.75

Los resultados evidencian que la aplicación logró mantenerse operativa durante la prueba de estrés, sin generar errores en las solicitudes ejecutadas. Sin embargo, los tiempos de respuesta aumentaron de forma considerable, especialmente en el acceso al panel administrativo, que presentó un promedio de 20147 ms.

Evaluación por Usuario Final

Para efectos de esta actividad, la evaluación por parte del usuario final se plantea como una simulación hipotética de prueba de aceptación. En un entorno real, esta validación debería ser realizada por un usuario final autorizado, como un médico, administrador de clínica o responsable del sistema, quien sería el encargado de revisar los flujos principales y dar el visto bueno formal sobre el funcionamiento de la aplicación.

Se realizó una simulación de evaluación de aceptación con el objetivo de representar la revisión que haría un usuario final sobre la aplicación web de gestión de pacientes clínicos. Esta simulación permitió verificar, de forma hipotética, si el sistema cumple con los flujos principales esperados para su uso básico.

Criterio evaluado	Resultado
El sistema permite iniciar sesión correctamente	Cumple
El administrador puede consultar información limitada de pacientes	Cumple
El doctor puede consultar información clínica completa	Cumple
El sistema bloquea accesos no autorizados	Cumple
La aplicación responde correctamente en los flujos principales	Cumple
La API responde correctamente ante solicitudes válidas y errores controlados	Cumple

De acuerdo con la revisión realizada, la aplicación cumple con los flujos principales esperados para la gestión básica de pacientes clínicos. El sistema permite diferenciar los permisos entre administrador y doctor, protege la información clínica sensible y responde correctamente ante accesos no autorizados.

Como oportunidad de mejora, se recomienda ampliar la aplicación con formularios completos para registro, búsqueda avanzada, actualización de pacientes y gestión detallada del historial médico, con el fin de ofrecer una experiencia más completa al usuario final.

Link GitHub

Link en GitHub para ver el proyecto:

<https://github.com/lrojas137/actividad-11-plan-pruebas-aceptacion>

Resultado de aprendizaje

Con el desarrollo de esta actividad se fortaleció la capacidad para diseñar, ejecutar y documentar pruebas de aceptación en una aplicación web. Se aplicaron herramientas como TestLink, Selenium, Postman y JMeter para validar casos de prueba, automatizar flujos de interfaz, probar servicios API, verificar permisos y evaluar el rendimiento del sistema.

Además, se comprendió la importancia de las pruebas de aceptación como una etapa clave antes de entregar una aplicación al usuario final. Estas pruebas permiten confirmar que el sistema cumple los requisitos funcionales, protege la información sensible y responde adecuadamente ante diferentes escenarios de uso.

Referencias

- Apache Friends. (s.f.). *XAMPP Installers and Downloads for Apache Friends*. Obtenido de Apache Friends: <https://www.apachefriends.org/es/index.html>
- Apache Software Foundation. (s.f.). *Apache JMeter User's Manual*. Obtenido de Apache JMeter: <https://jmeter.apache.org/usermanual/index.html>
- Composer. (s.f.). *Composer: A dependency manager for PHP*. Obtenido de <https://getcomposer.org/>
- Laravel. (s.f.). *Testing: Getting started*. Obtenido de Laravel Documentation 12.x.: <https://laravel.com/docs/12.x/releases>
- PHPUnit. (s.f.). *Documentation for PHPUnit*. Obtenido de <https://phpunit.de/documentation.html>
- Postman. (s.f.). *Postman Learning Center*. Obtenido de <https://learning.postman.com/>
- Selenium. (s.f.). *The Selenium Browser Automation Project*. Obtenido de <https://www.selenium.dev/documentation/>
- TRMS, T. O. (s.f.). *TestLink documentation*. Obtenido de <https://testlink.org/>